GPU Architectures 1

Agenda

- Understand the GPU execution model
  - SIMT (vs. SIMD)

- Learn the details of GPU architectures
  - NVIDIA GPU architecture
  - AMD GPU architecture
  - ARM GPU architecture

Motivation

◎ Why do we discuss vendor-specific hardware?

- **Understand important design decisions in the recent history of GPGPU evolution**
  - Observe similarities and differences between GPUs from different vendors
  - Project what to expect in the future

- **Gain intuition into how to write efficient kernel codes and optimize them**
  - GPU kernels directly determine how program's loops and data are mapped to the underlying hardware
  - Understanding hardware will allow for platform-specific tuning of codes

- **Write performance-portable kernel codes**
  - Many GPU programming models are platform-independent (e.g., OpenCL, OpenMP, …)

Introduction to GPU Architectures

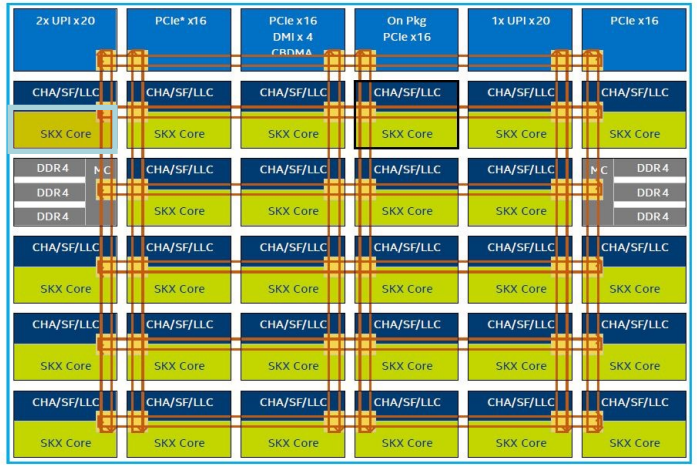
Review: CPU vs. GPU

- Different goals produce different designs
  - GPU assumes workloads are highly parallel
  - CPU must be good at everything, parallel or not

- CPU: minimize latency experienced by one thread
  - Big on-chip caches
  - Sophisticated control logic

- GPU: maximize throughput of all threads
  - # of thread in flight limited by resources → lots of resources (registers, BW, etc.)
  - Multithreading can hide latency → skip the big caches
  - Shared control logic across many threads

CPU core



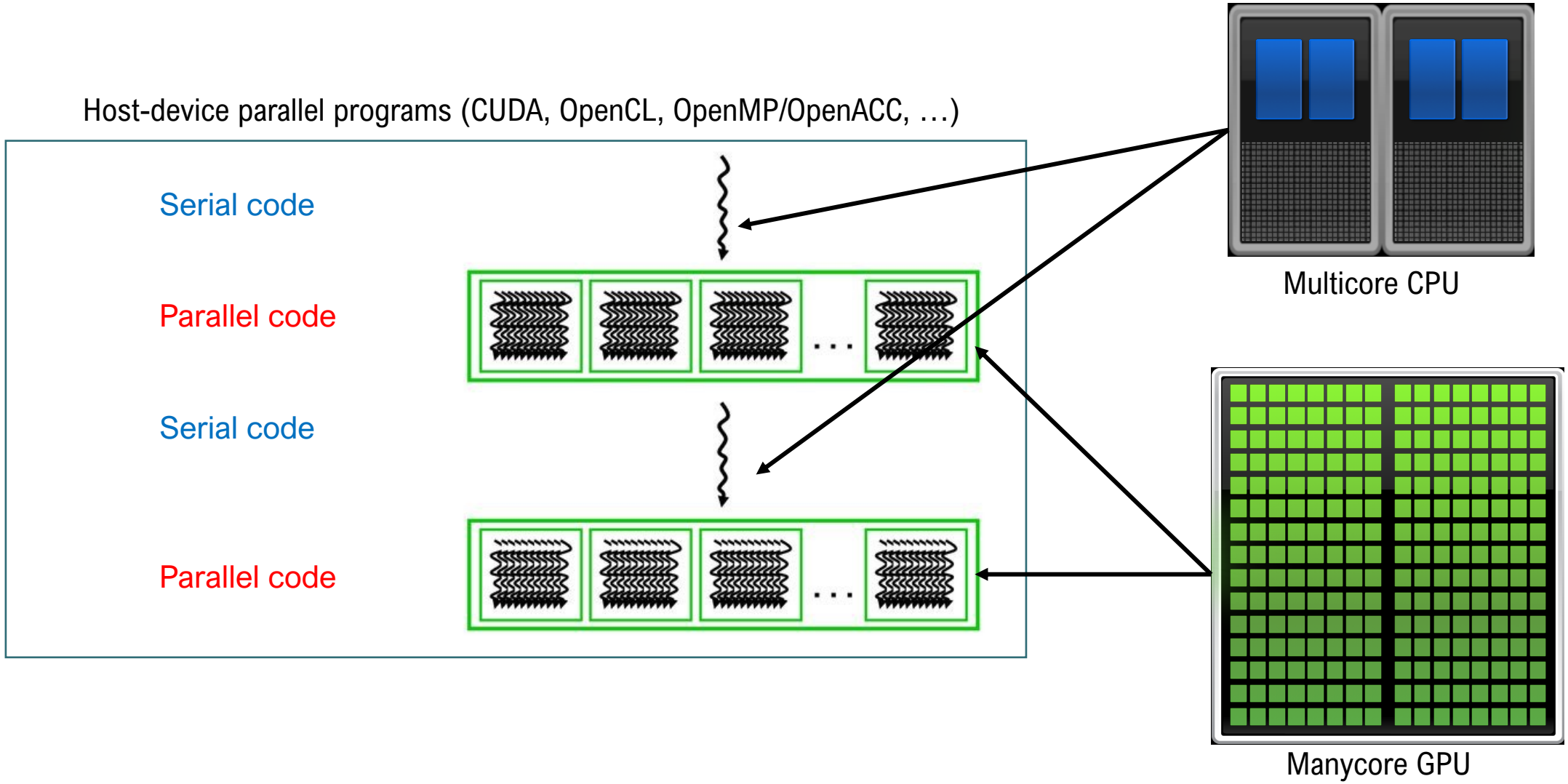
Intel Skylake Xeon CPU (28 cores)

SM (Simultaneous Multiprocessor)



NVIDIA Volta GPU
(5120 CUDA cores, 640 Tensor cores)

Review: Heterogeneous Computing

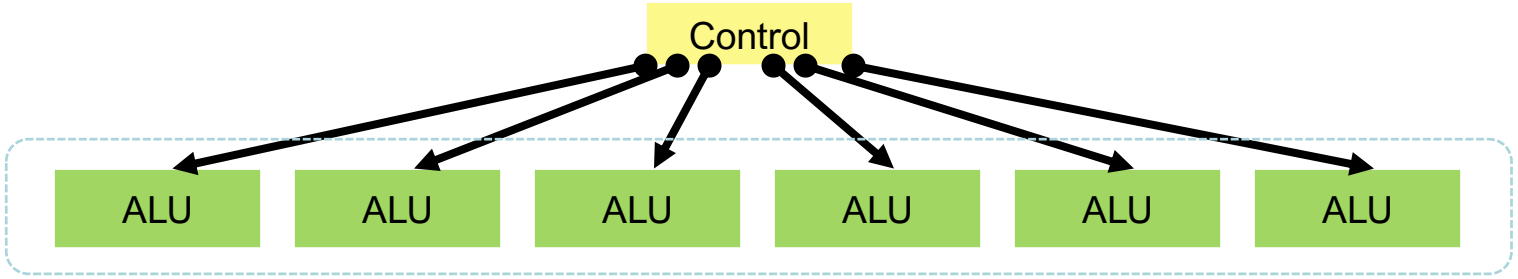


Higher Efficiency

- Note that GPU performs similar operations for lots of pixels/vertices
  - Major source of parallelism
- (Multi-core) general-purpose processor:



- GPU's efficiency comes from minimizing the "control" overhead

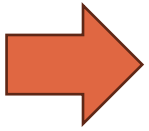


Warp/Wavefront = A set of threads executing the same instruction
A warp is essentially a SIMD operation formed by hardware!

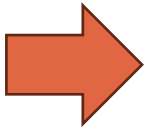
Review: SIMD

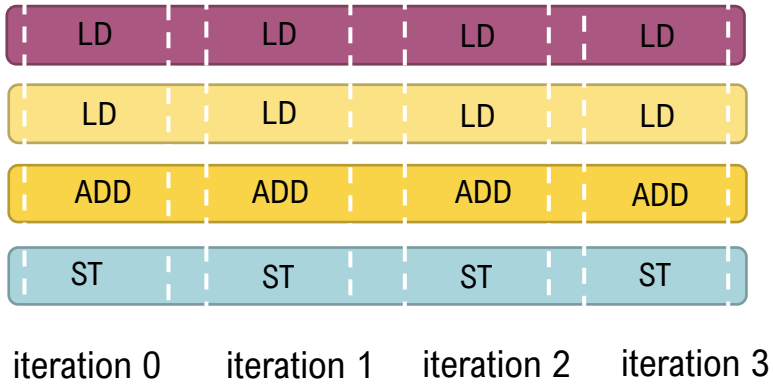
- A single sequential stream of SIMD (vector) instructions
  - Each SIMD instruction specifies multiple data points
- Both programming model and execution model are SIMD
- Need to know the SIMD width
  - To generate a correct sequence of vector instructions
  - Vectorization by programmers or compilers

```
for (i=0; i<N; i++)
     c[i] = a[i] + b[i]
```



```
VLD v0 = a
VLD v1 = b
VADD v2 = v0 + v1
VST c = v2
```





GPUs are SIMD Engines Underneath

- The instruction pipeline operates like a SIMD pipeline (e.g., an array processor)

- However, the programming is done using threads, NOT SIMD instructions

→ Separation of programming model (software) and execution model (hardware)

Programming Model vs. Hardware Execution Model

- Programming Model refers to how the programmer expresses the code
  - E.g., Sequential (von Neumann), Data Parallel (SIMD), Dataflow, Multi-threaded (MPMD, SPMD), …

- Execution Model refers to how the hardware executes the code underneath
  - E.g., Out-of-order execution, Vector processor, Array processor, Dataflow processor, Multiprocessor, Multithreaded processor, …

- Execution Model can be very different from the programming model
  - E.g., von Neumann model implemented by an OoO processor
  - E.g., SPMD model implemented by a SIMD processor (a GPU)

SIMD Machine with SPMD Programming Model

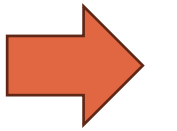
- **S**ingle **P**rogram, **M**ultiple **D**ata
  - This is a programming model rather than computer organization
  - A subclass of MIMD
    - SPMD: Multiple instruction streams but the same instruction at each step
    - MPMD: Multiple instruction streams and different instructions
  - A coarse-grained form of SIMD

- Each thread executes the same “kernel”, but operates a different piece of data
  - Each thread has its own context (i.e., can be treated/restarted/executed independently)
  - Each program/procedure can execute a different control-flow path at run-time
  - Many scientific applications are programmed this way and run on MIMD hardware (multiprocessors)

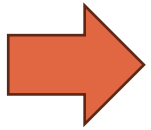
SIMT

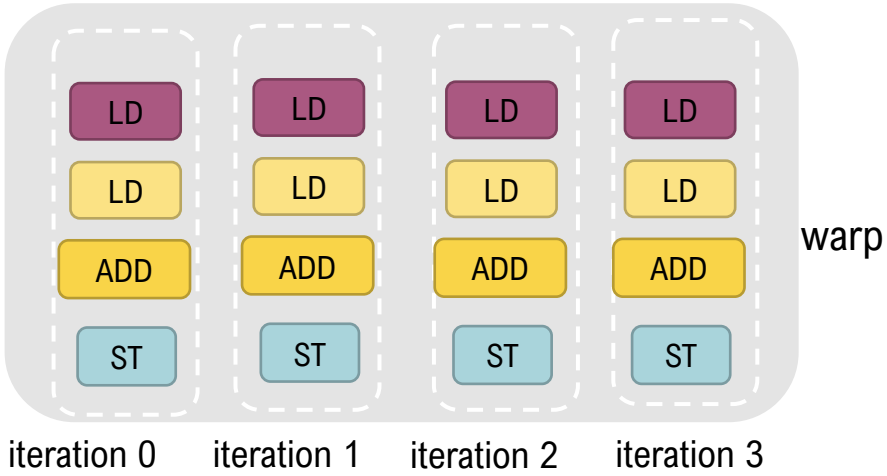
- Single Instruction, Multiple Threads (coined by NVIDIA)
- Multiple instruction streams of scalar instructions
  - Threads grouped dynamically into warps
- A set of threads executing the same instruction are dynamically grouped into a warp (wavefront) by the hardware
  - A warp is essentially a SIMD operation formed by hardware

```
(i=0; i<N; i++)
 c[i] = a[i] + b[i]
```



```
LD v0 = a[id]
LD v1 = b[id]
ADD v2 = v0 + v1
ST c = v2[id]
```





SIMD vs. SIMT Execution Model

◎ SIMD: A single sequential instruction stream of SIMD instructions → each instruction specifies multiple data inputs

- [VLD, VLD, VADD, VST], VLEN

◎ SIMT: Multiple instruction streams of scalar instructions → threads grouped dynamically into warps

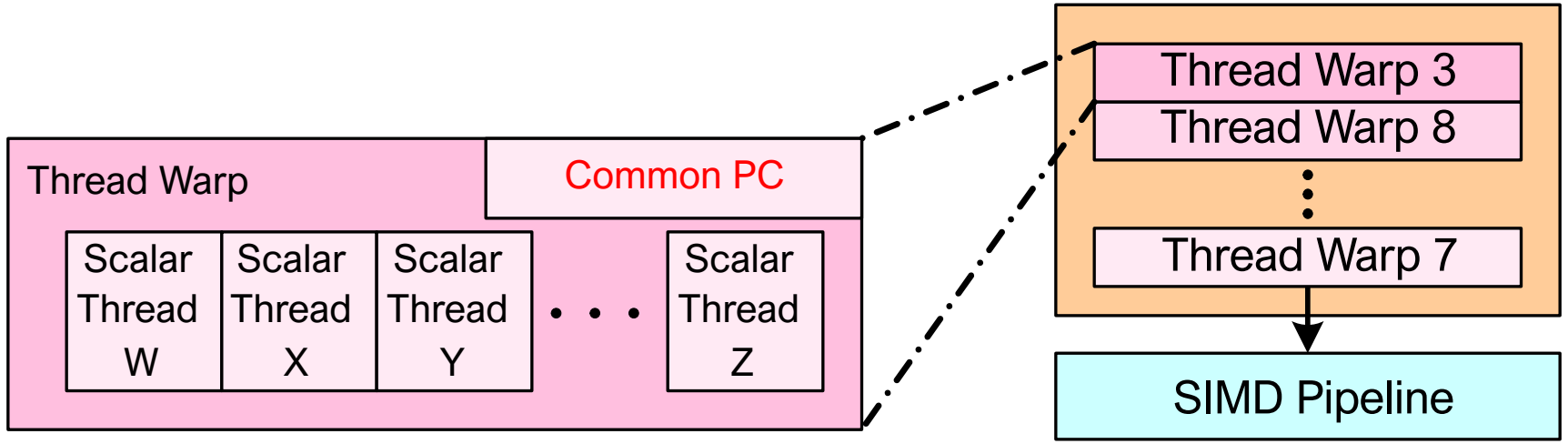
- [LD, LD, ADD, ST], NumThreads

◎ Two Major SIMT Advantages:

- Can treat each thread separately → i.e., can execute each thread independently (on any type of scalar pipeline) → MIMD processing
- Can group threads into warps flexibly → i.e., can group threads that are supposed to truly execute the same instruction → dynamically obtain and maximize benefits of SIMD processing

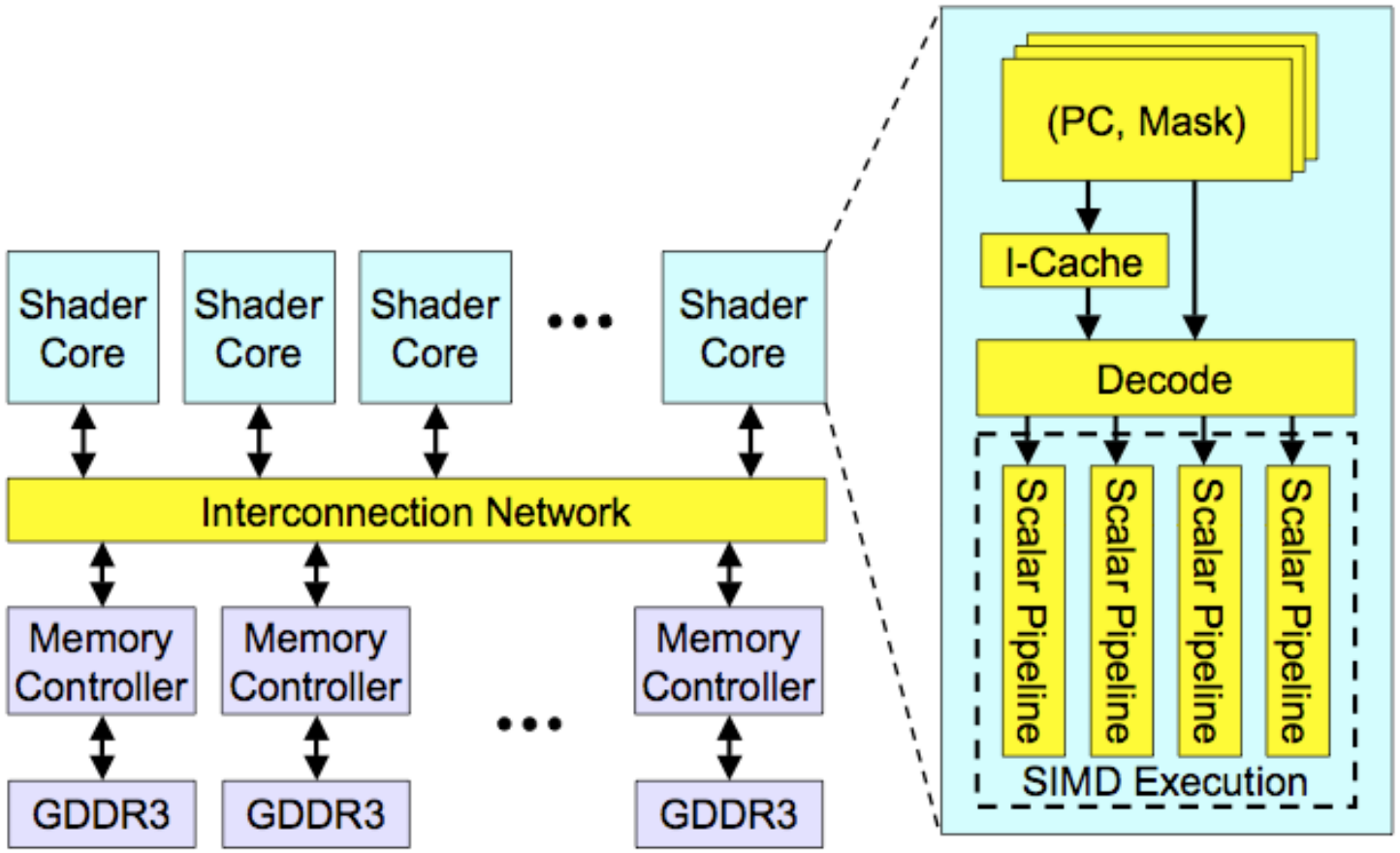
Warps and Warp-Level FGMT

- Warp: A set of threads that execute the same instruction (on different data elements)
  - A warp is the threads that run lengthwise in a woven fabric
- All threads in a warp share the PC (and call stack)
  - Each thread with its own registers
  - Threads in a different warp, even for the same program, may have different PC



Lindholm et al., "NVIDIA Tesla: A Unified Graphics and Computing Architecture," IEEE Micro 2008.

High-Level View of a GPU



Lindholm et al., "NVIDIA Tesla: A Unified Graphics and Computing Architecture," IEEE Micro 2008.

Latency Hiding via Warp-Level FGMT

◎ Warp: A set of threads that execute the same instruction (on different data elements)

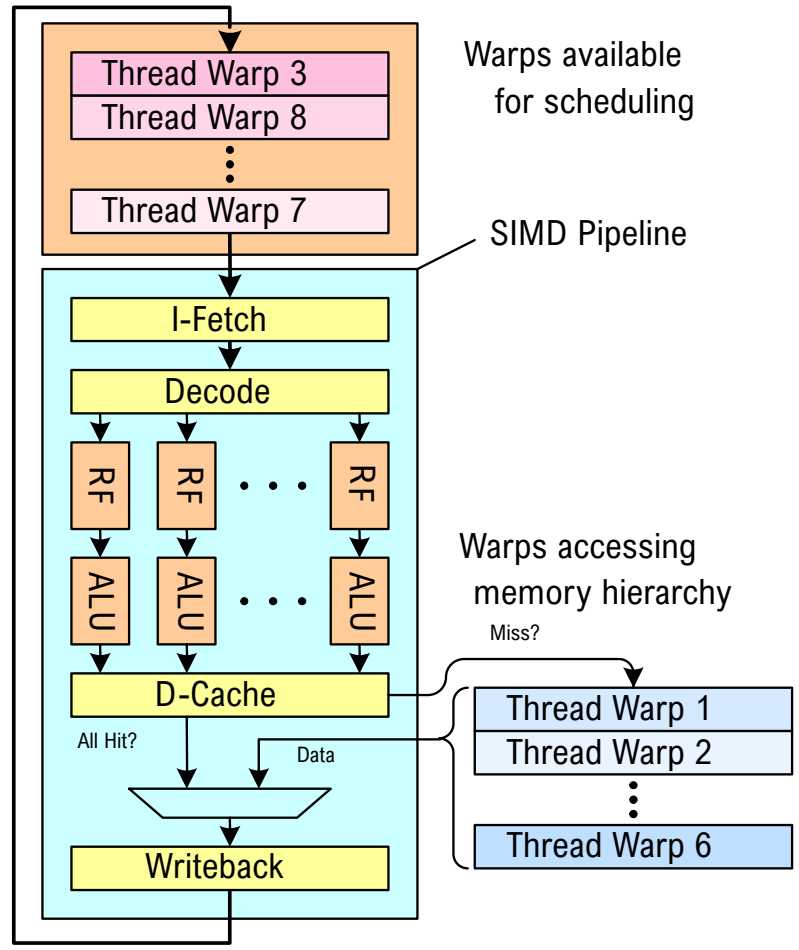
◎ Fine-grained multithreading

- One instruction per thread in pipeline at a time (No interlocking)
- Interleave warp execution to hide latencies

◎ Register values of all threads stay in register file

◎ FGMT enables long latency tolerance

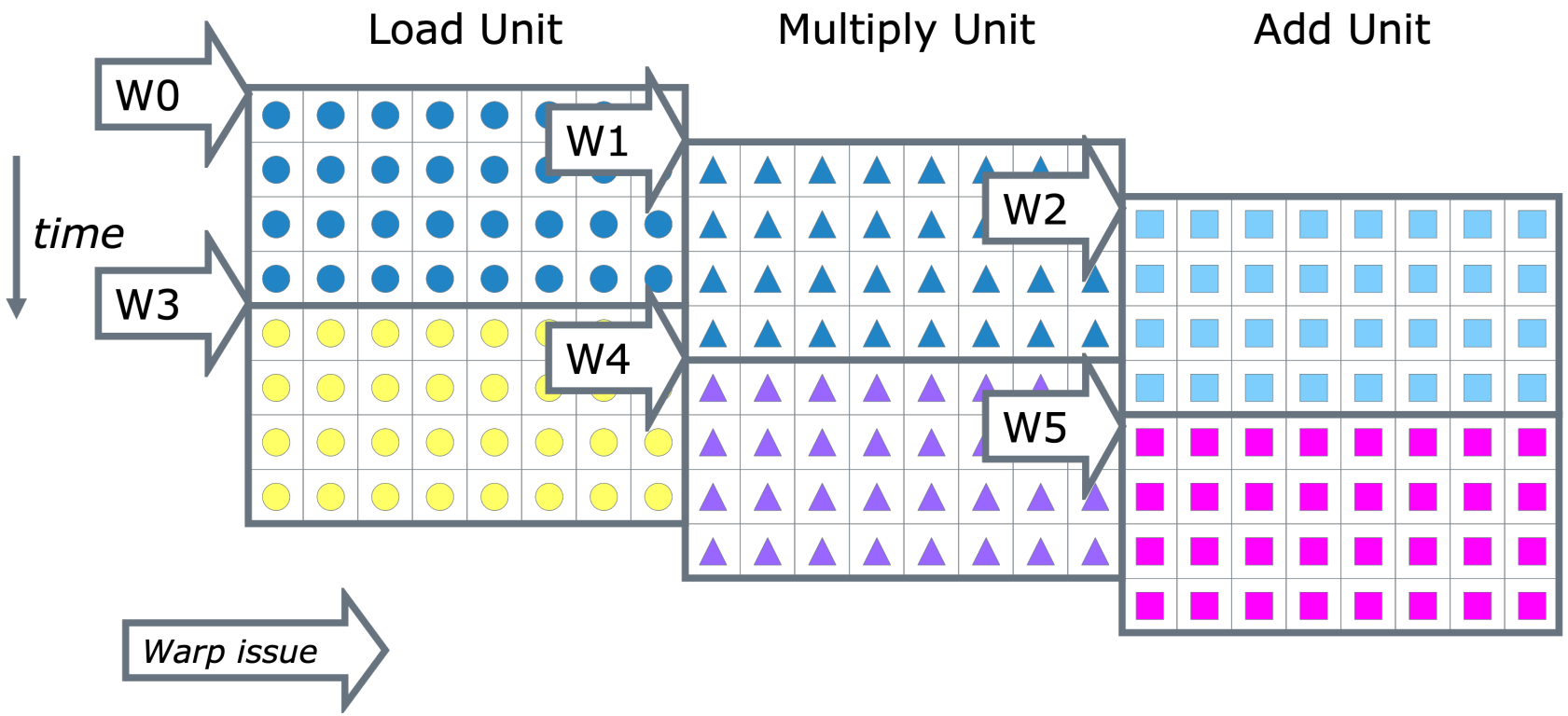
- Millions of pixels



Warp Instruction Level Parallelism

Can overlap execution of multiple instructions

- Example machine has 32 threads per warp and 8 lanes
- Completes 24 operations/cycle while issuing 1 warp/cycle



Sample GPU SIMT Code (Simplified)

CPU code

```
for (ii = 0; ii < 100000; ++ii) {
C[ii] = A[ii] + B[ii];
}
```



CUDA code

```
// there are 100000 threads
__global__ void KernelFunction(…) {
  int tid = blockDim.x * blockIdx.x + threadIdx.x;
  int varA = aa[tid];
  int varB = bb[tid];
  C[tid] = varA + varB;
}
```

SIMD vs. SIMT Execution Model

◎ SIMD: A single sequential instruction stream of SIMD instructions → each instruction specifies multiple data inputs

- [VLD, VLD, VADD, VST], VLEN

◎ SIMT: Multiple instruction streams of scalar instructions → threads grouped dynamically into warps

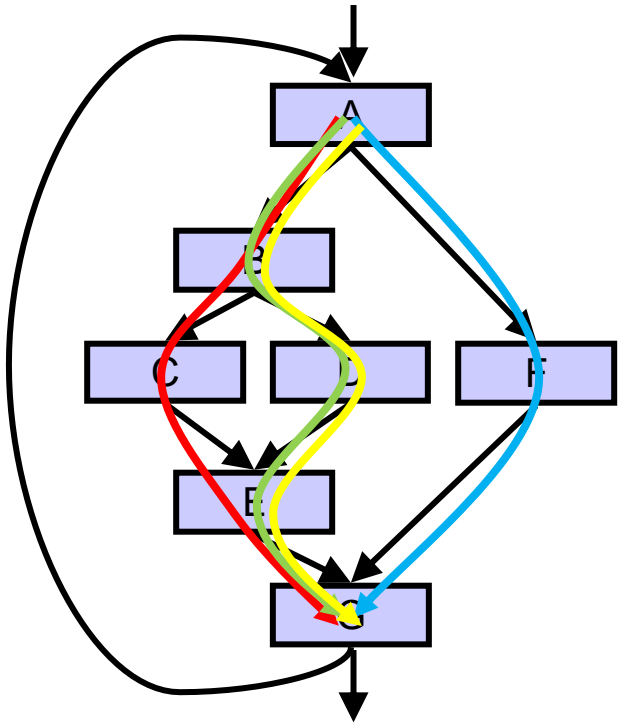
- [LD, LD, ADD, ST], NumThreads

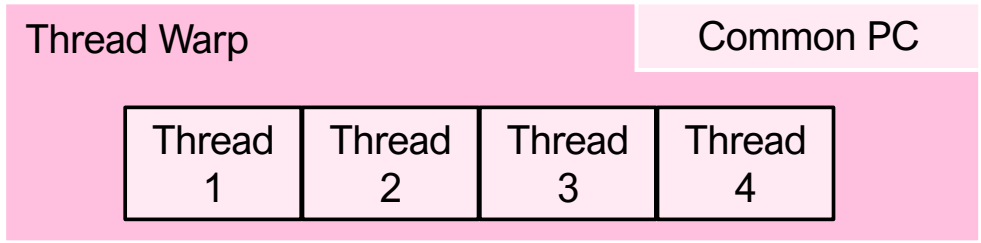
◎ Two Major SIMT Advantages:

- Can treat each thread separately → i.e., can execute each thread independently (on any type of scalar pipeline) → MIMD processing
- Can group threads into warps flexibly → i.e., can group threads that are supposed to truly execute the same instruction → dynamically obtain and maximize benefits of SIMD processing

Threads Can Take Different Paths in Warp-based SIMD

- Each thread can have conditional control flow instructions
- Threads can execute different control flow paths





Control Flow Problem in GPUs/SIMT

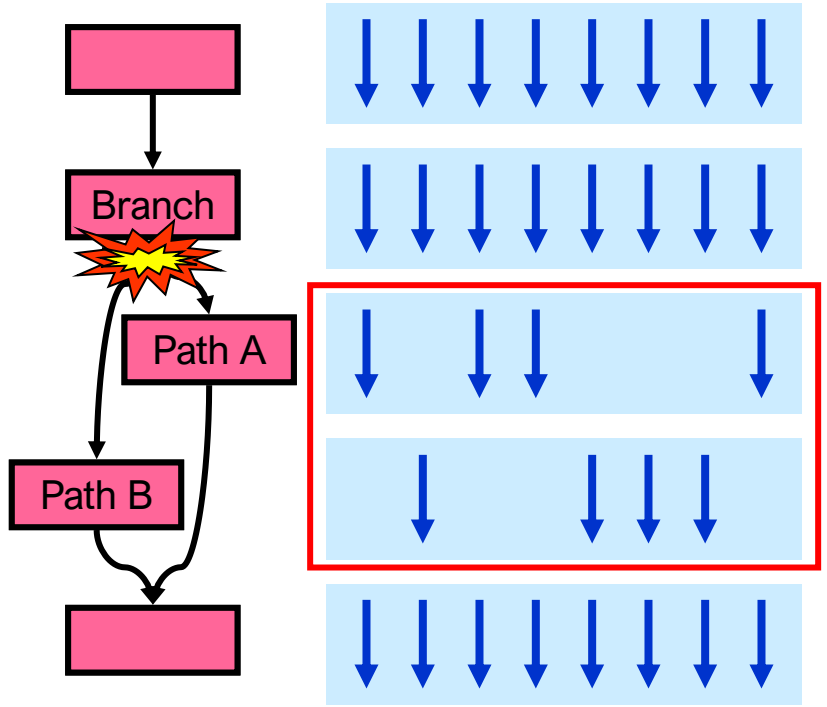
◎ A GPU uses a SIMD pipeline to save area on control logic

- Groups scalar threads into warps

◎ Branch divergence occurs when threads inside warps branch to different execution paths

◎ "Active mask" specifies which threads of the warp are active at any given time

- Serializes execution for different parts of the warp



Conditional/predicated/masked execution
C.f., Vector Mask and Masked Vector Operations

Remember: Each Thread Is Independent

- Two Major SIMT Advantages:
  - Can treat each thread separately → i.e., can execute each thread independently on any type of scalar pipeline → MIMD processing
  - Can group threads into warps flexibly → i.e., can group threads that are supposed to truly execute the same instruction → dynamically obtain and maximize benefits of SIMD processing

- If we have many threads
  - We can find individual threads that are at the same PC
  - And group them together into a single warp dynamically
  - This reduces "divergence" → Improves SIMD utilization
    - SIMD utilization: fraction of SIMD lanes executing a useful operation (i.e., executing an active thread)

- C.f., independent thread scheduling since NVIDIA Volta GPUs

Warp-based SIMD vs. Traditional SIMD

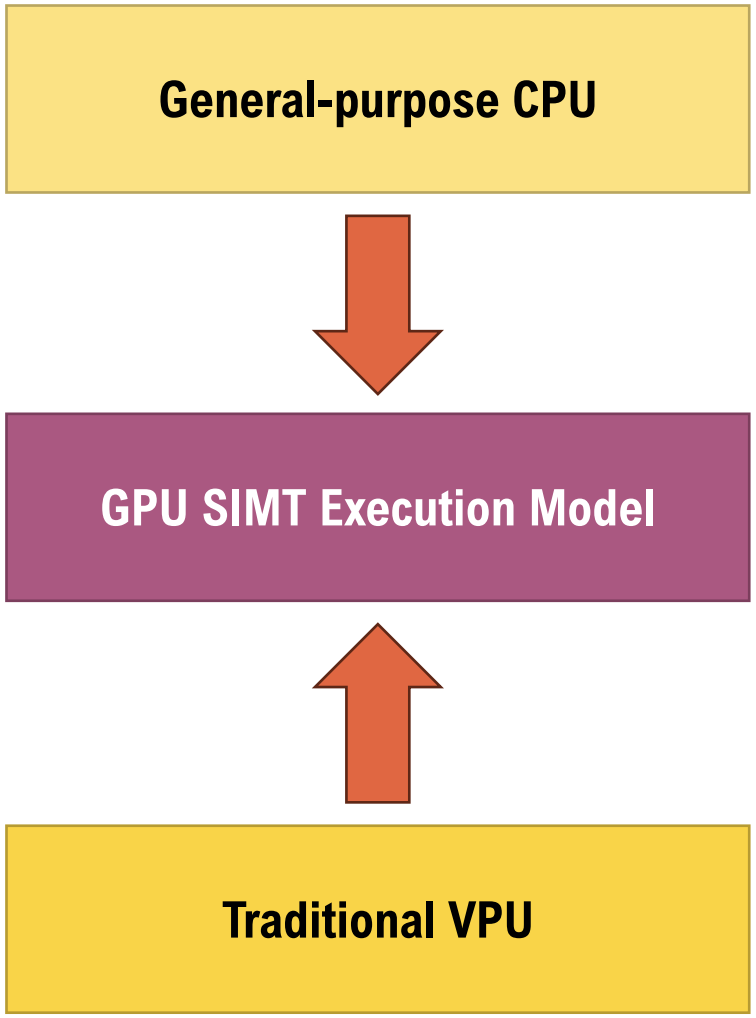
◎ Traditional SIMD contains a single thread

- Sequential instruction execution; lock-step operations in a SIMD instruction
- Programming model is SIMD (no extra threads) → SW needs to know vector length
- ISA contains vector/SIMD instructions

◎ Warp-based SIMD consists of multiple scalar threads executing in a SIMD manner (i.e., same instruction executed by all threads)

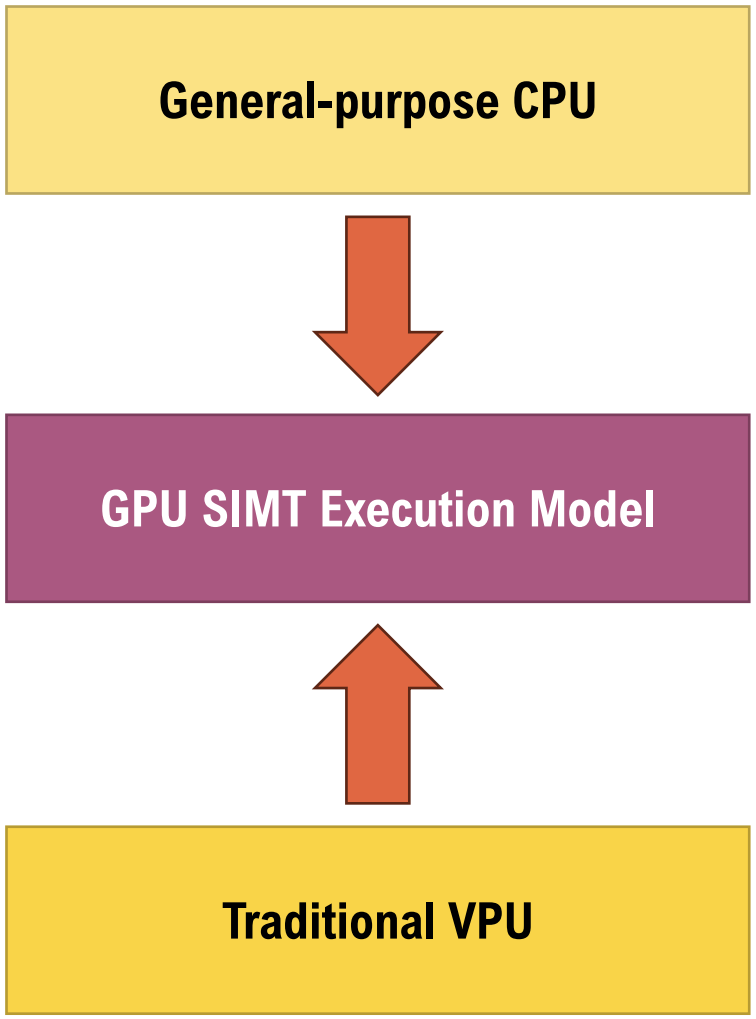
- Does not have to be lock step
- Each thread can be treated individually (i.e., placed in a different warp) → programming model not SIMD
  - SW does not need to know vector length
  - Enables multithreading and flexible dynamic grouping of threads
- ISA is scalar → SIMD operations can be formed dynamically
- Essentially, it is SPMD programming model implemented on SIMD hardware

Summary



Optimize hardware for throughput on data-parallel workloads
→ Retain SPMD programming model for portability and programmability
→ Execute threads in groups that share instruction control
→ Replace complicated control logic with lots of simpler SIMD hardware

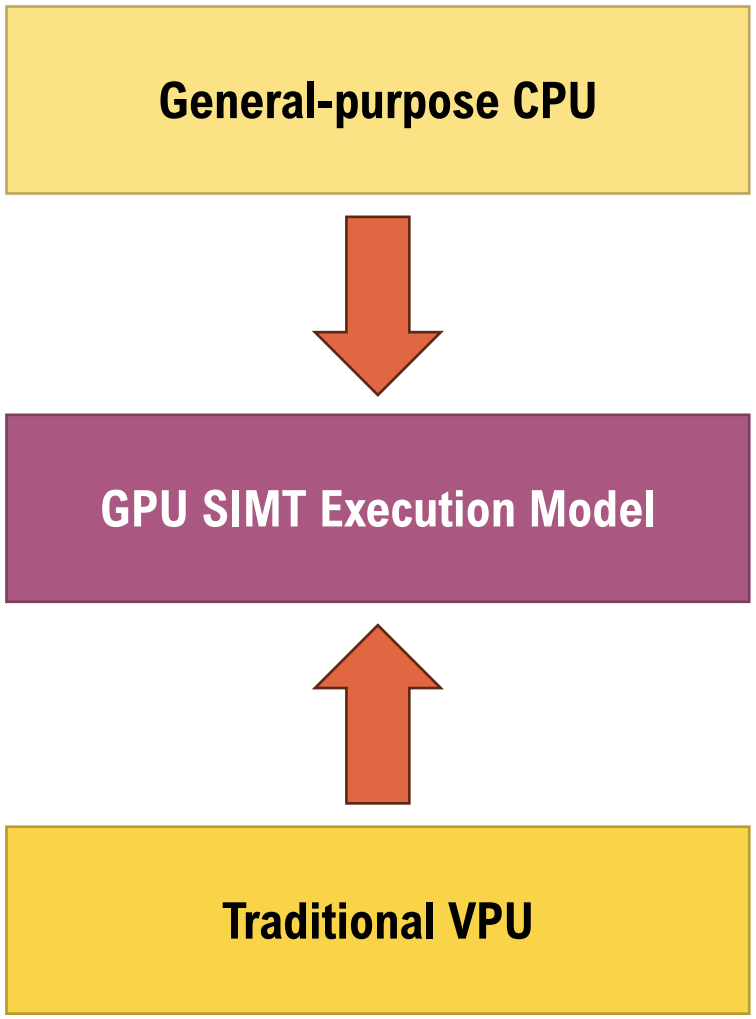
Summary



Optimize hardware for throughput on data-parallel workloads
→ Retain SPMD programming model for portability and programmability
→ Execute threads in groups that share instruction control
→ Replace complicated control logic with lots of simpler SIMD hardware

Makes SIMD hardware multi-threaded; schedule coarse-grained units (programs) in a SIMD manner
→ Scheduler interleaves different groups of SIMD executions with enough # of available work
→ More programmable and efficient way of exploiting scaled-out resources
→ Can support code divergence with predication

Summary



Optimize hardware for throughput on data-parallel workloads
→ Retain SPMD programming model for portability and programmability
→ Execute threads in groups that share instruction control
→ Replace complicated control logic with lots of simpler SIMD hardware

DLP-oriented, TLP-assisted, ILP-enabled

Makes SIMD hardware multi-threaded; schedule coarse-grained units (programs) in a SIMD manner
→ Scheduler interleaves different groups of SIMD executions with enough # of available work
→ More programmable and efficient way of exploiting scaled-out resources
→ Can support code divergence with predication

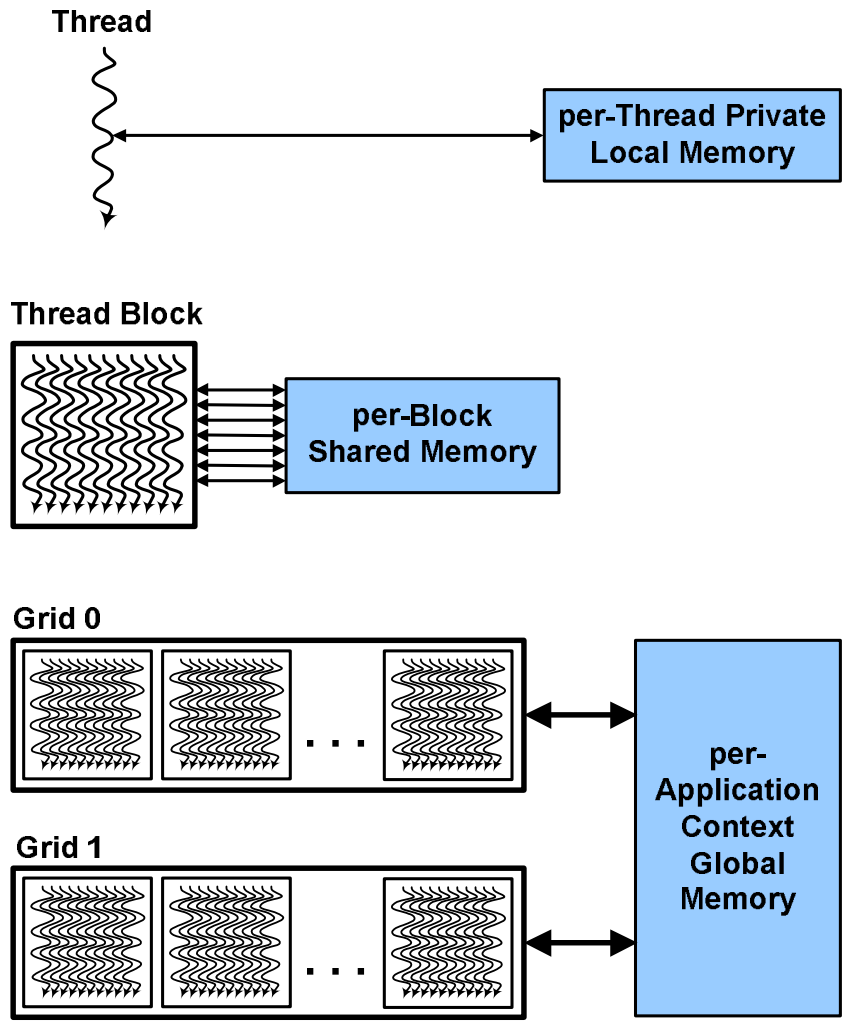
NVIDIA GPU Architectures

Terminology

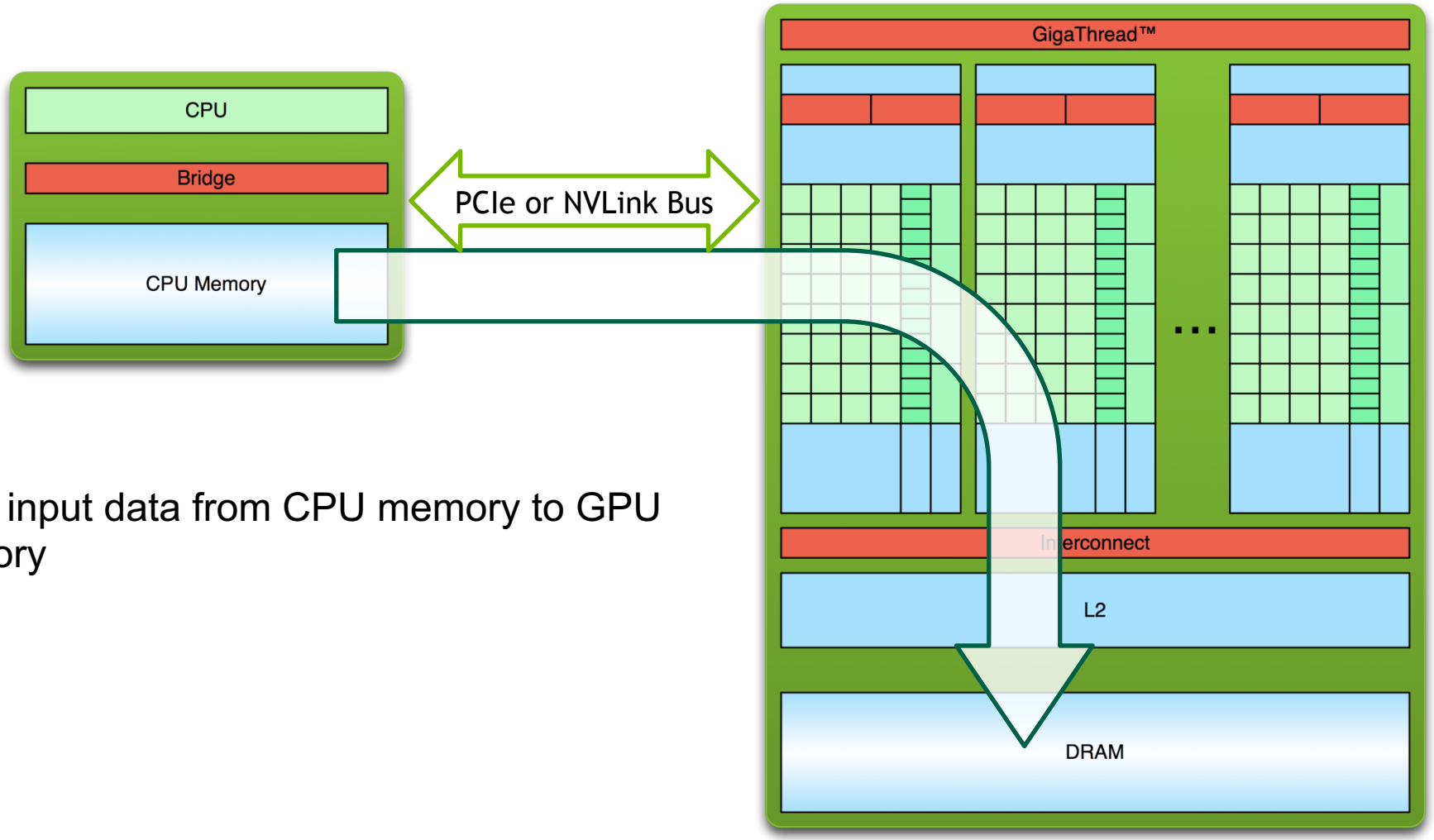
NVIDIA/CUDA Terminology	AMD Terminology	Description	
Kernel	Kernel	Functions launched to the GPU that are executed by multiple threads on the GPU. Kernels can work in parallel with CPU.	Software concepts
Thread Block	Work-group	A group of work-items/threads that execute the same “kernel” on different data. Executed as groups of warps/wavefronts on a single SM/CU. Can synchronize together and communicate through local memory.	
Thread	Work-item / Thread	Individual thread of execution in a warp/wavefront.	
Streaming Multiprocessor (SM)	Compute Unit (CU)	One of many parallel vector processors in a GPU that contain parallel ALUs.	Hardware concepts
Warp	Wavefront	Collections of operations that execute in lock-step, run the same instructions, and follow the same control-flow paths. Individual lanes can be masked off. They are vectorized hardware threads.	

Mapping CUDA to NVIDIA Memory Hierarchy

- Thread: an instance of the kernel
  - Thread ID within the thread block, PC, registers, per-thread private memory, inputs, and output results
- Thread block: a set of concurrently executing threads that can cooperate among them
  - Communicate through barriers and shared memory
  - Block ID within the grid
- Grid: an array of thread blocks that execute the same kernel
  - Read from and write to global memory
  - Global synchronization



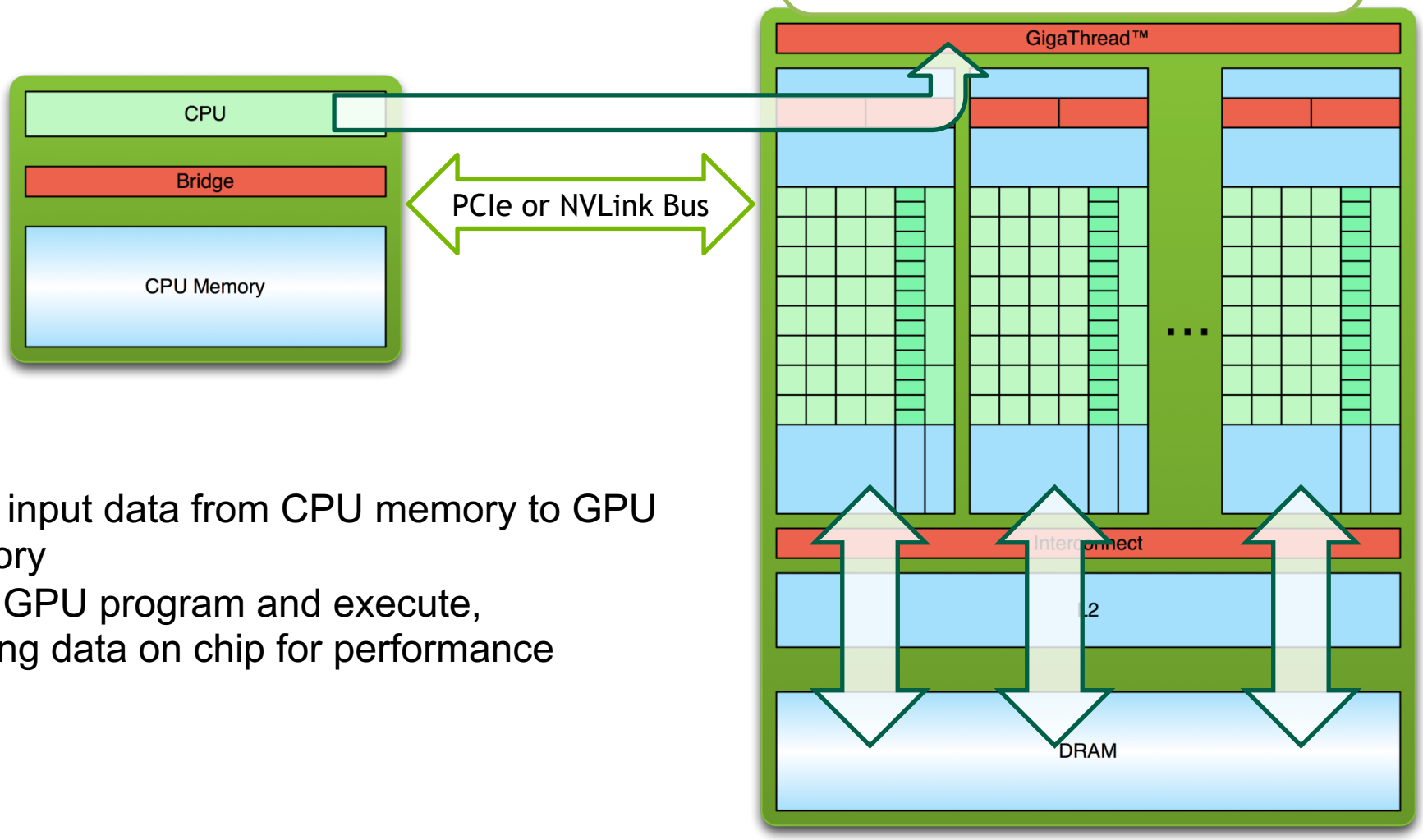
Processing Flow



1. Copy input data from CPU memory to GPU memory

Processing Flow

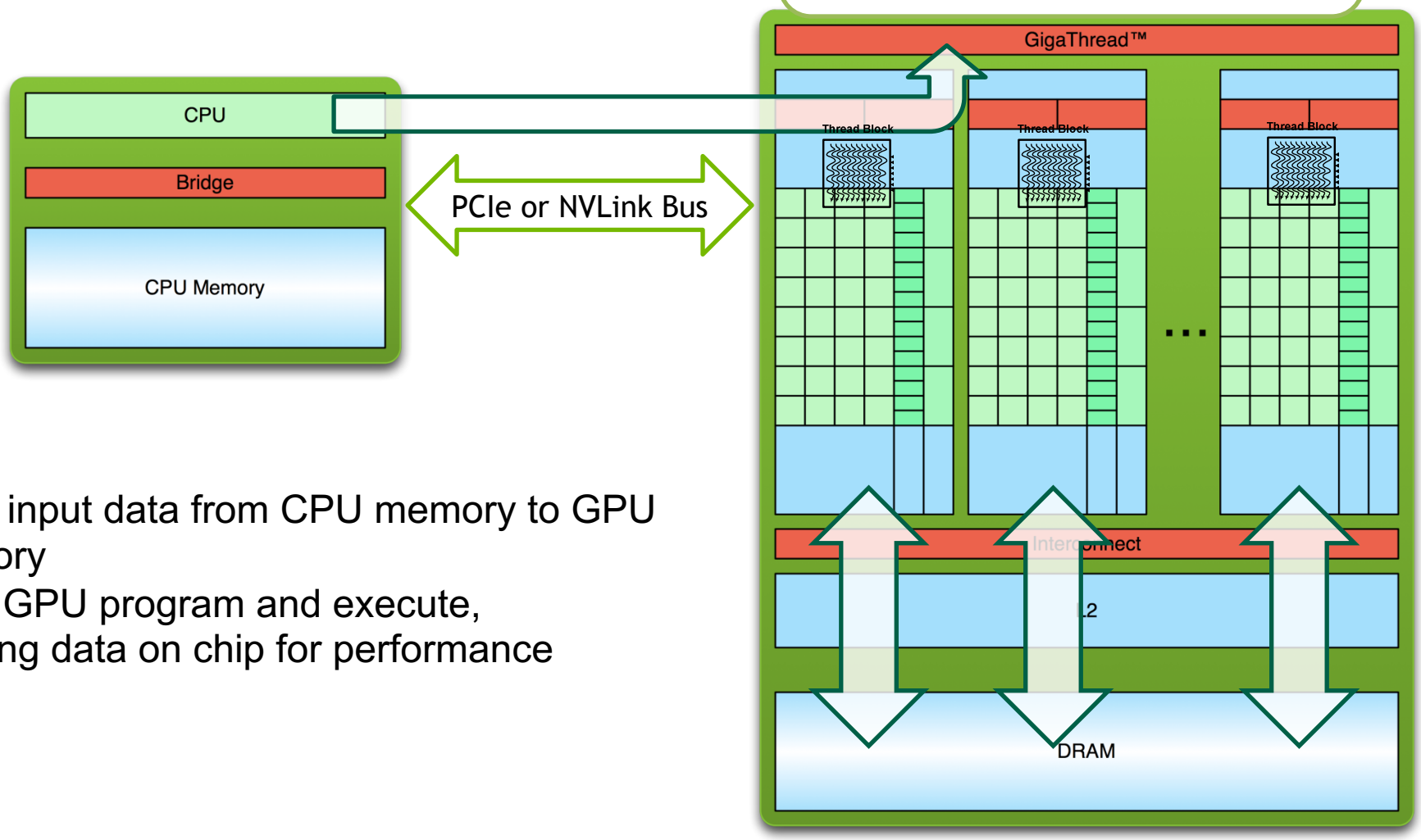
```
// there are 100000 threads
__global__ void KernelFunction(…) {
  int tid = blockDim.x * blockIdx.x + threadIdx.x;
  int varA = aa[tid];
  int varB = bb[tid];
  C[tid] = varA + varB;
}
```



1. Copy input data from CPU memory to GPU memory
2. Load GPU program and execute, caching data on chip for performance

Processing Flow

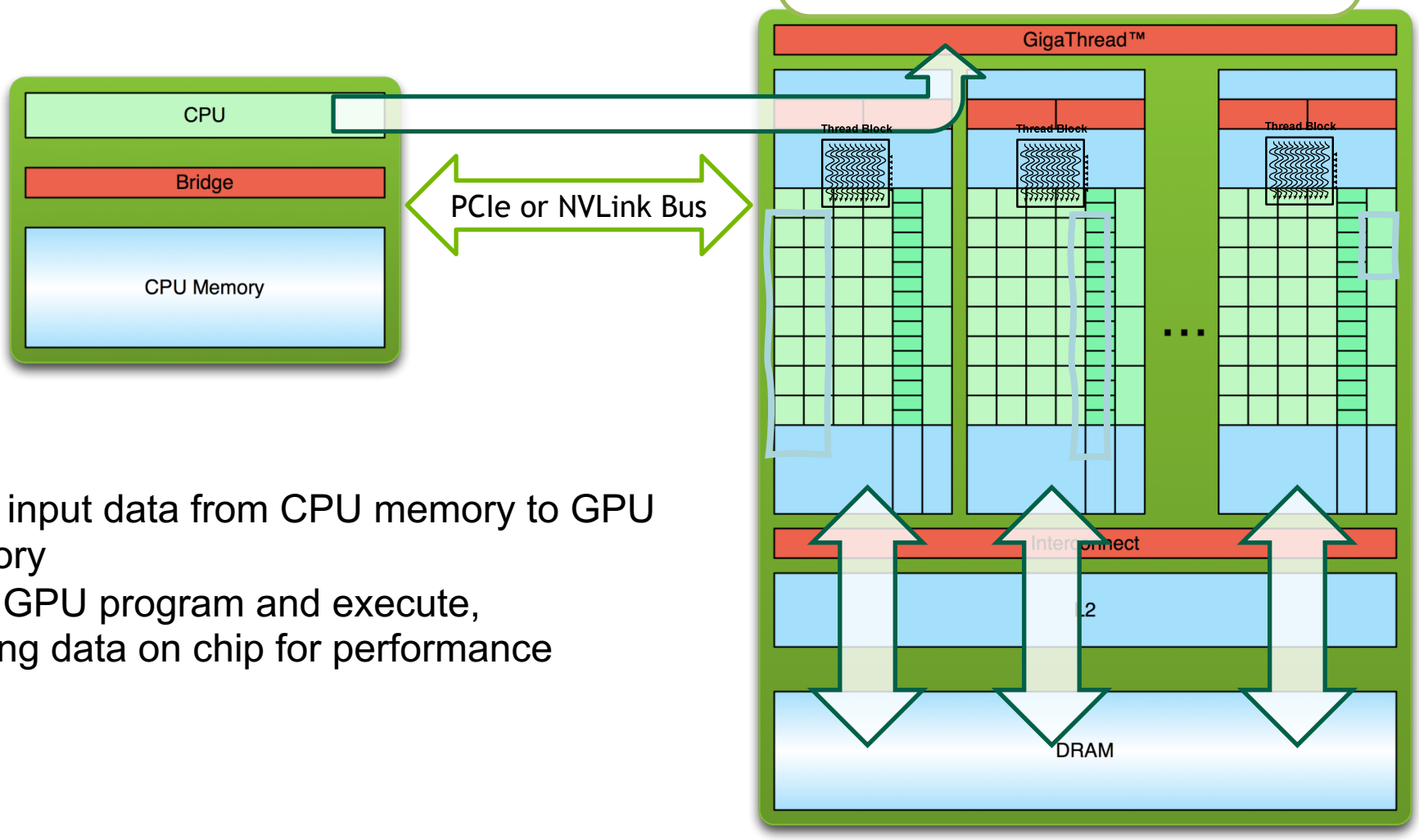
```
// there are 100000 threads
__global__ void KernelFunction(…) {
  int tid = blockDim.x * blockIdx.x + threadIdx.x;
  int varA = aa[tid];
  int varB = bb[tid];
  C[tid] = varA + varB;
}
```



1. Copy input data from CPU memory to GPU memory
2. Load GPU program and execute, caching data on chip for performance

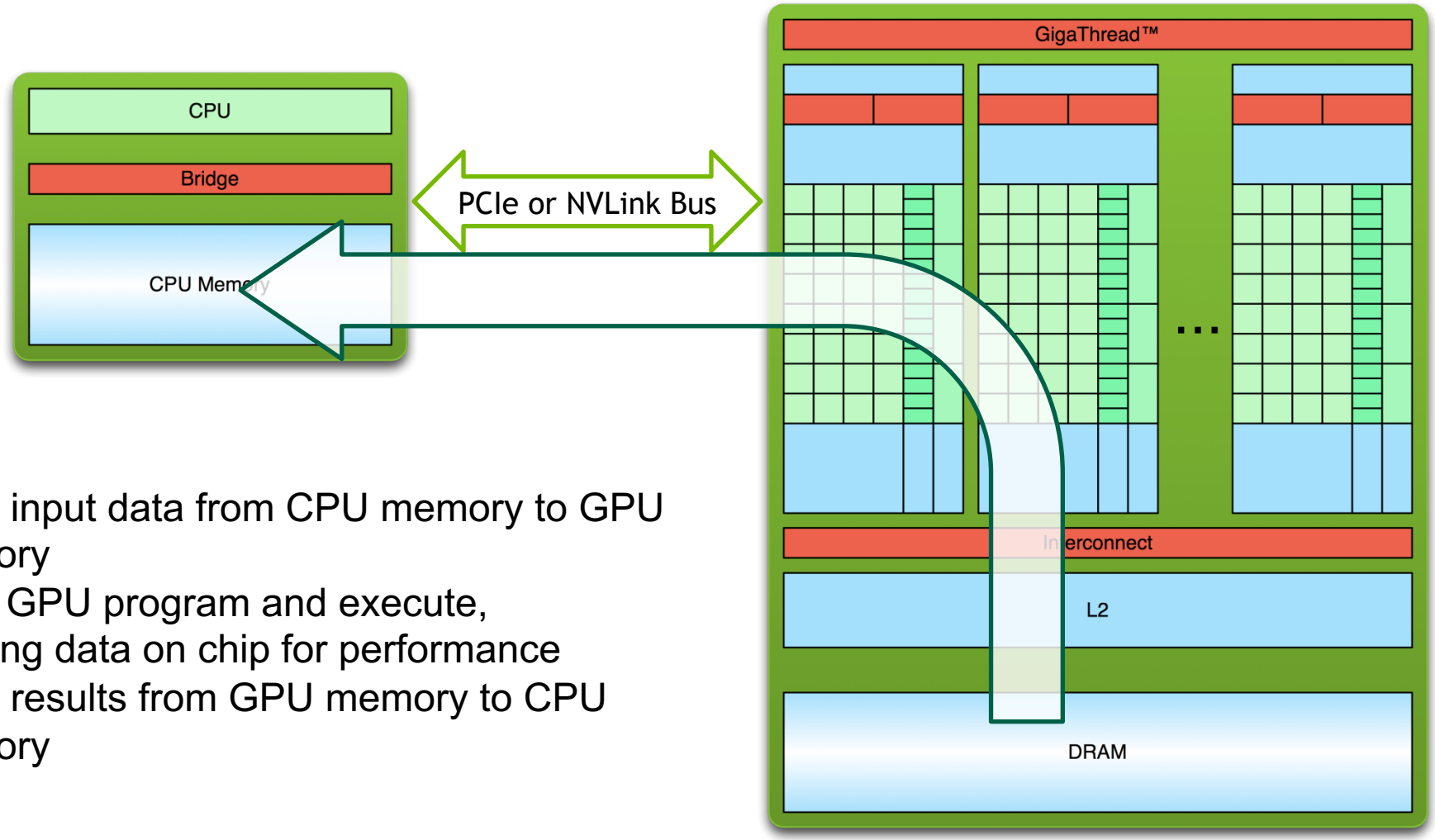
Processing Flow

```
// there are 100000 threads
__global__ void KernelFunction(…) {
  int tid = blockDim.x * blockIdx.x + threadIdx.x;
  int varA = aa[tid];
  int varB = bb[tid];
  C[tid] = varA + varB;
}
```



1. Copy input data from CPU memory to GPU memory
2. Load GPU program and execute, caching data on chip for performance

Processing Flow



1. Copy input data from CPU memory to GPU memory
2. Load GPU program and execute, caching data on chip for performance
3. Copy results from GPU memory to CPU memory

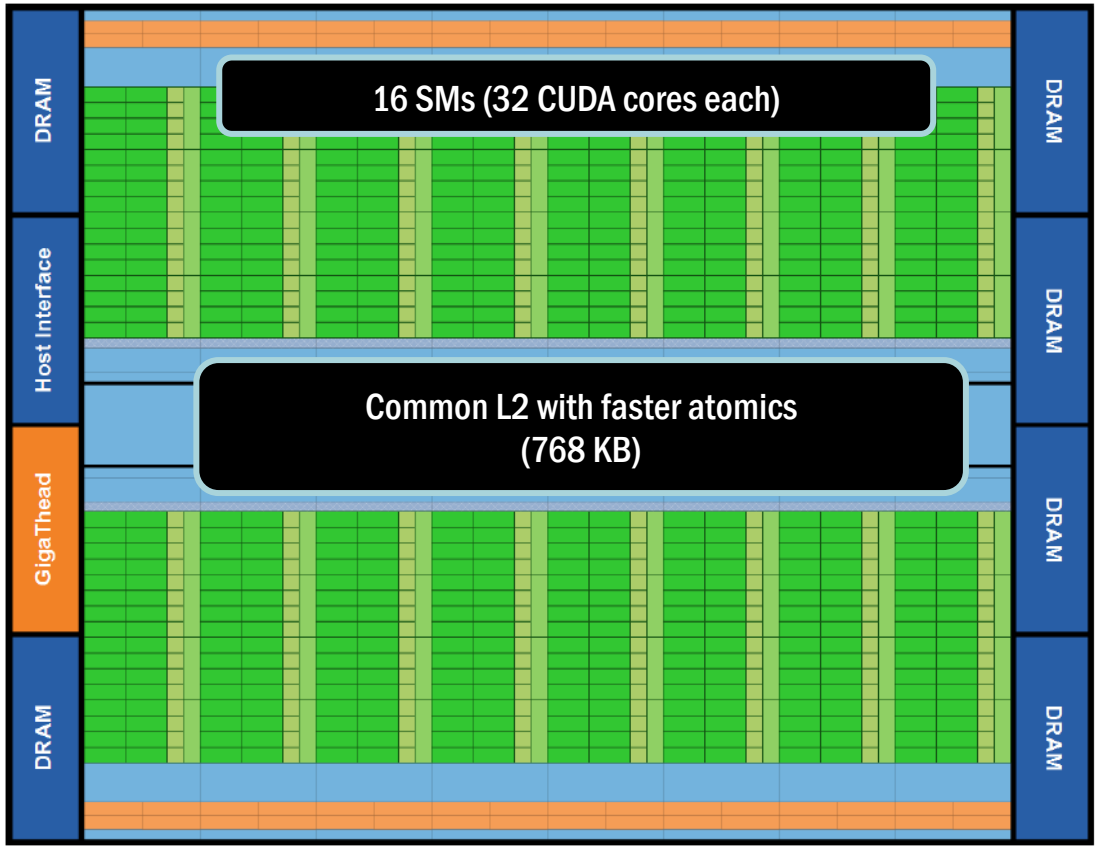
Features Across Generations

- G80-Pascal (2006-2016)
  - Establishing the foundation for GPU computing
    - Introduced unified shaders for general-purpose compute with caches, atomics, and concurrent kernels
    - Pascal added HBM2, unified memory with hardware page faults, and the first NVLink generation
- Volta-Ampere (2017-2020)
  - Adding specialized execution units around the SIMT SM's
    - Tensor Cores (Volta) and RT Cores (Turing)
- Hopper (2022)
  - Expanding asynchronous execution and cooperation across SMs
    - Tensor memory accelerator (TMA) and thread-block clusters with distributed shared memory
- Blackwell-Rubin (2024-2026)
  - Increasing emphasis on system-scale computing
    - Co-design CPUs, GPUs, and networking through platforms such as Grace Blackwell and Vera Rubin

NVIDIA Fermi

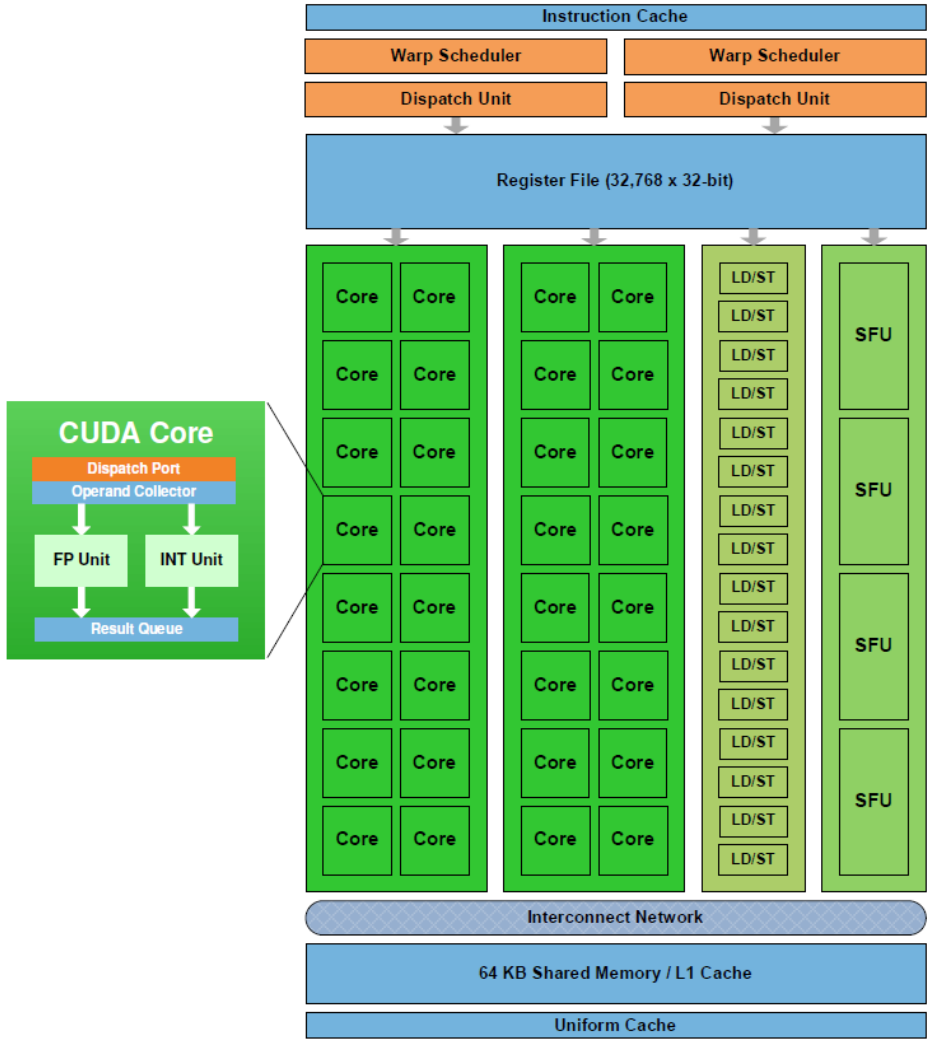
- G80 (2006)
  - The first GPU that introduced SIMT execution model with a unified processor for graphics and general computation

- Fermi: GF110 (2011)
  - 512 cores from 16 SMs
  - 1.3 TFlops (FP32), 177 GB/s memory BW



NVIDIA Fermi: SM

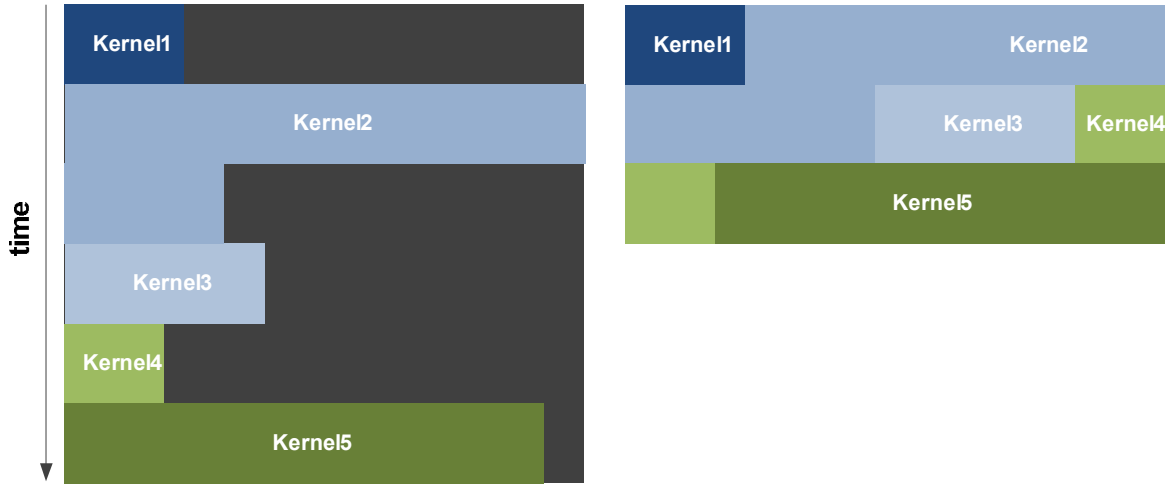
- Scalar register-based ISA
- 2 hardware warp schedulers
  - 32 threads per warp
- 2 instruction dispatch units
- 4 execution blocks
  - 2 groups of 16 CUDA cores
    - A CUDA core has ALU and FPU
  - 4 SFUs: Special function units
    - `sin`, `cos`, `sqrt`, `rcp` (reciprocal)
  - 16 load/store units



Fermi Streaming Multiprocessor (SM)

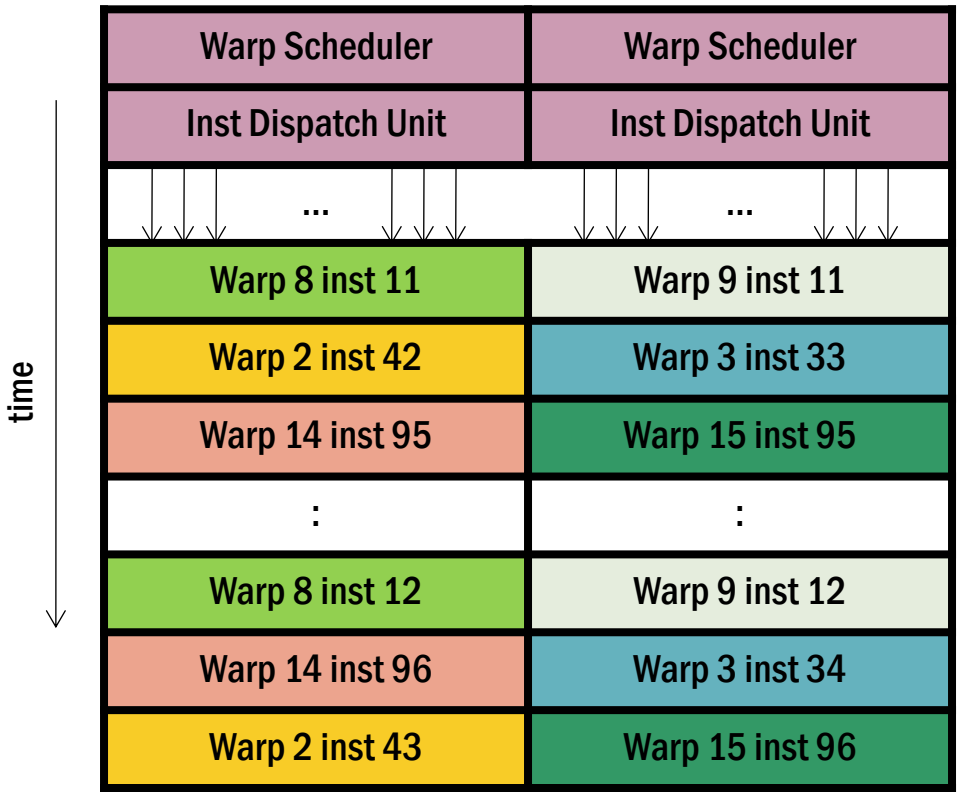
Thread Scheduler

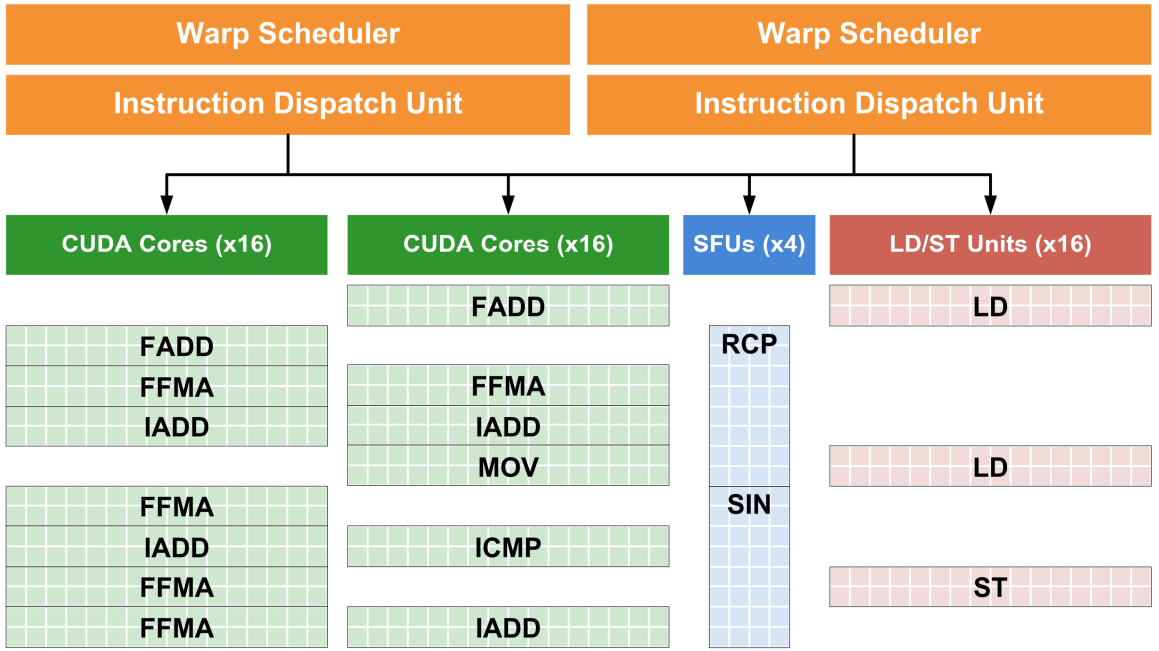
- Two-level thread schedulers
  - Chip level: global work distribution engine schedules thread blocks to various SMs
  - SM level: warp scheduler distributes warps to its execution units
- Supports concurrent kernel execution



NVIDIA Fermi: SM

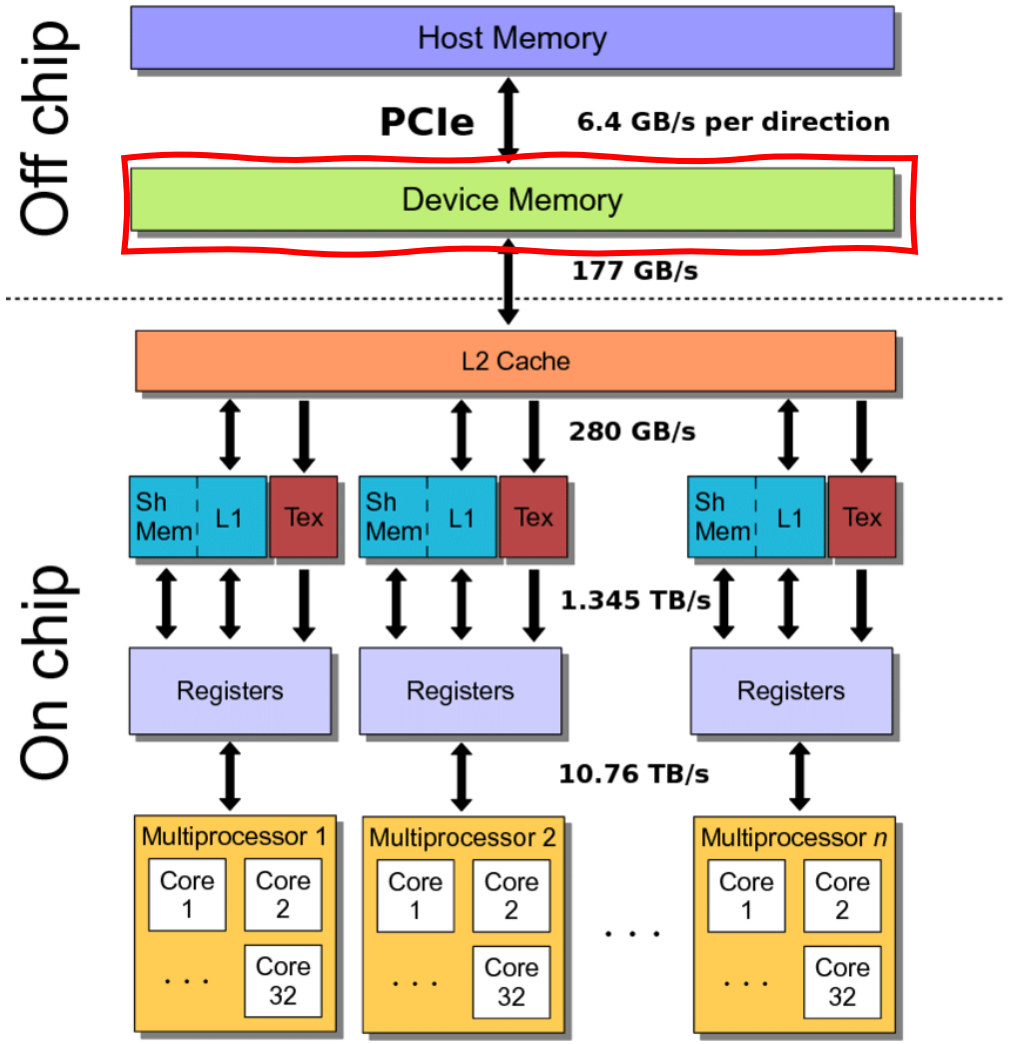
- Two instructions across active warps issued and executed concurrently
  - One instruction from a warp issued to an execution block
- Warps from different thread blocks or kernels can be scheduled on the same SM





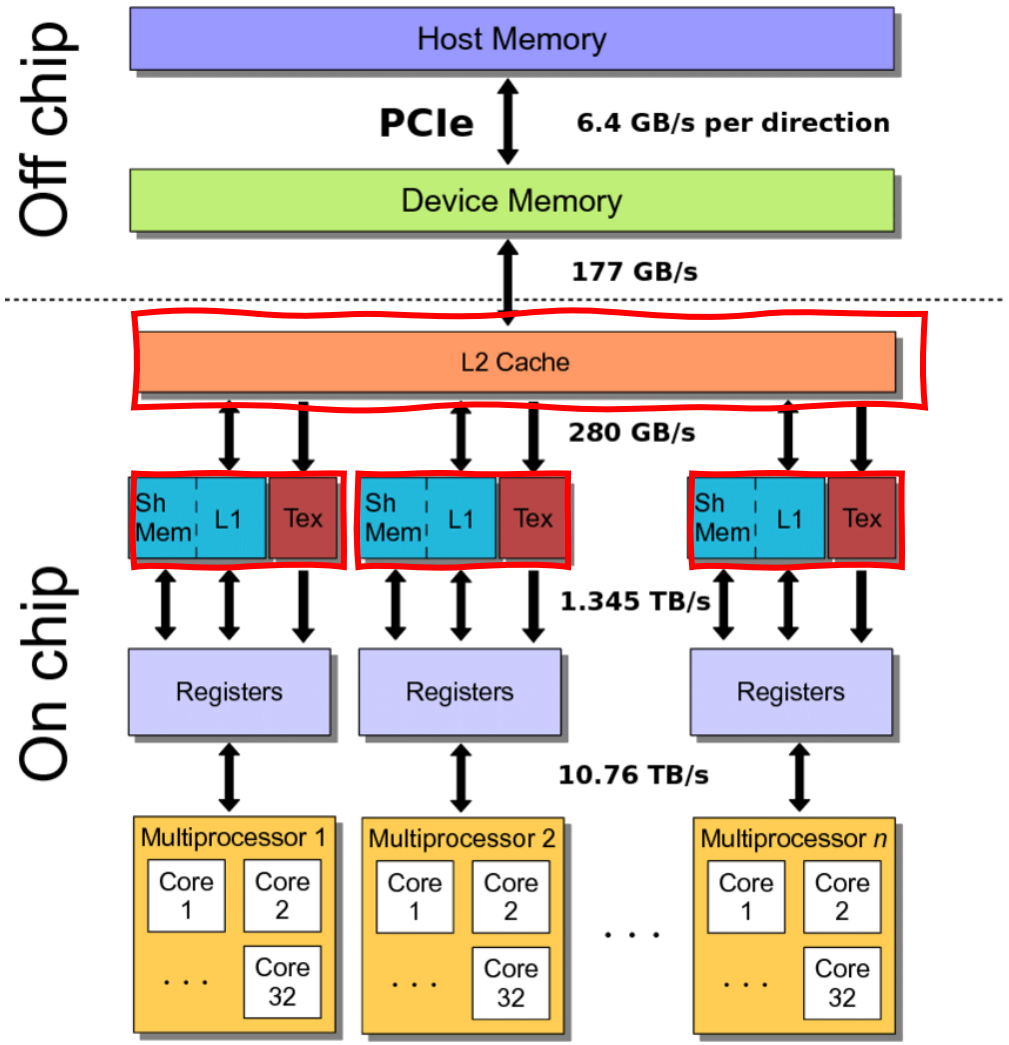
Fermi Memory Hierarchy

- Global memory
  - Main GPU memory (GDDR5)
  - Communicates with host
  - Can be seen by all threads
  - Up to 6GB
  - 400-800 cycle latency



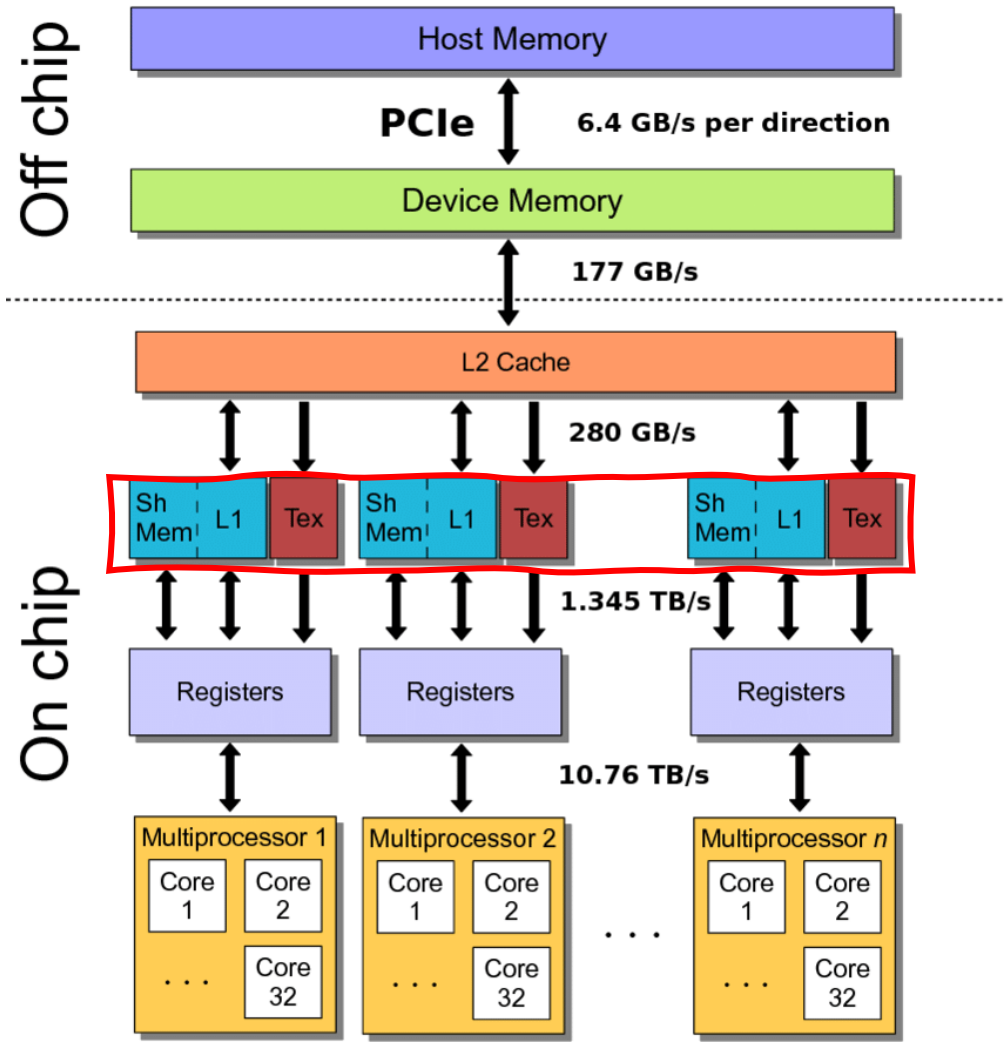
Fermi Memory Hierarchy

- L2 cache
  - Services all operations (load, store, texture)
  - Shared by all threads
  - Supports fast atomic memory operations
  - 768KB



Fermi Memory Hierarchy

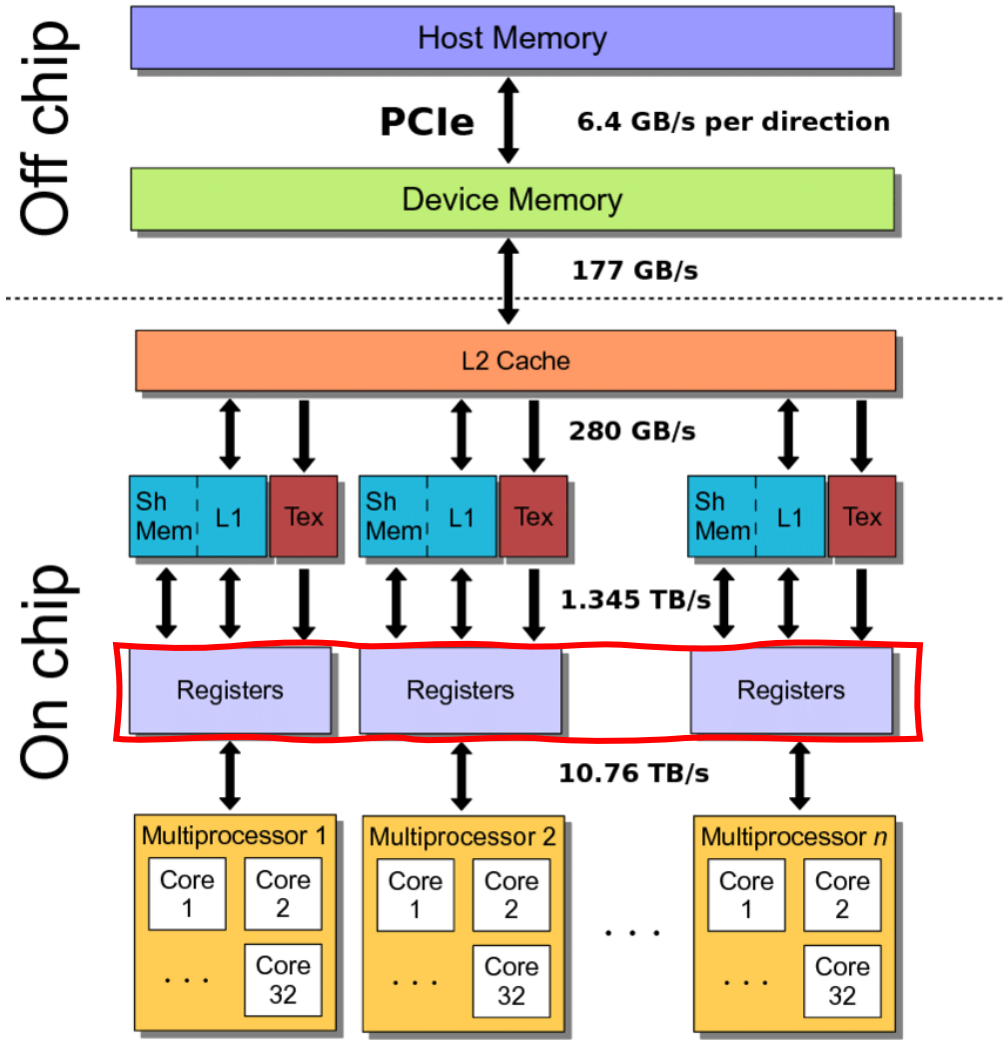
- 64KB L1 cache per SM
  - Can be configured to support shared memory and caching of global memory
  - 48KB shared memory + 16KB L1 cache or 16KB shared memory + 48KB L1 cache
  - 10-30 cycle latency
  - Shared memory acts as a user-managed cache to reduce global accesses
    - Read: global → registers → shared
    - Write: shared → registers → global
- 64KB constant cache per SM
- 12KB texture cache per SM



Fermi Memory Hierarchy

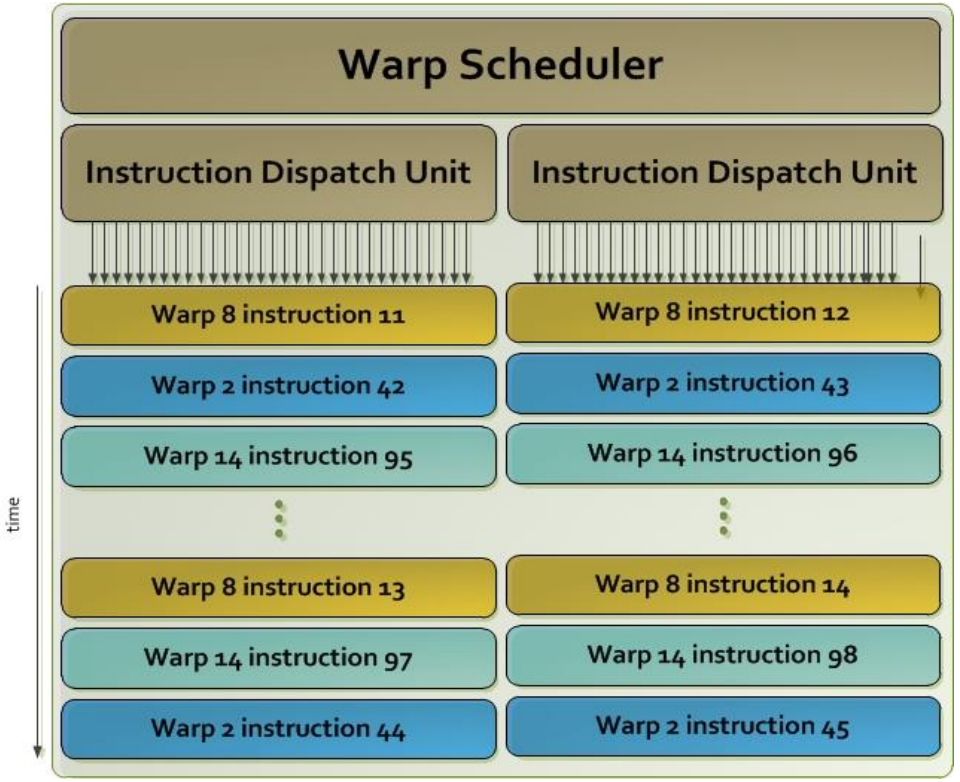
- Registers per SM
  - Private to each thread
  - Large, unified register file
  - 128KB (2MB across the chip)
  - 48 active warps per SM

  → 21 registers per thread



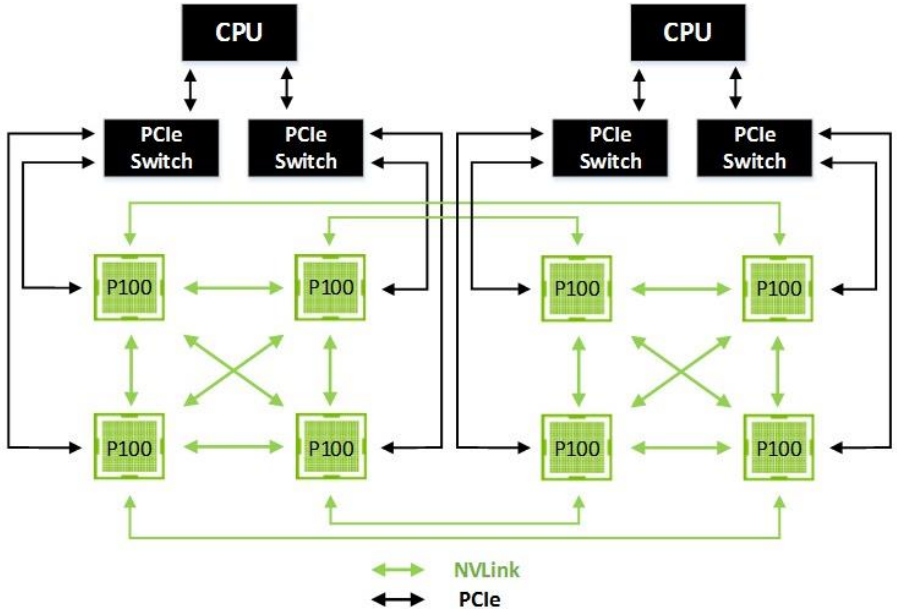
Kepler (2014)

- Increased double precision FP performance for HPC
- 4 warp schedulers, each with dual instruction dispatch units
  - Two independent instructions per warp can be dispatched each cycle
- Texture units with read-only caches
- Dynamic parallelism
  - Allows the GPU to generate new work for itself
  - Synchronize on results
  - Control the scheduling of that work without involving the CPU



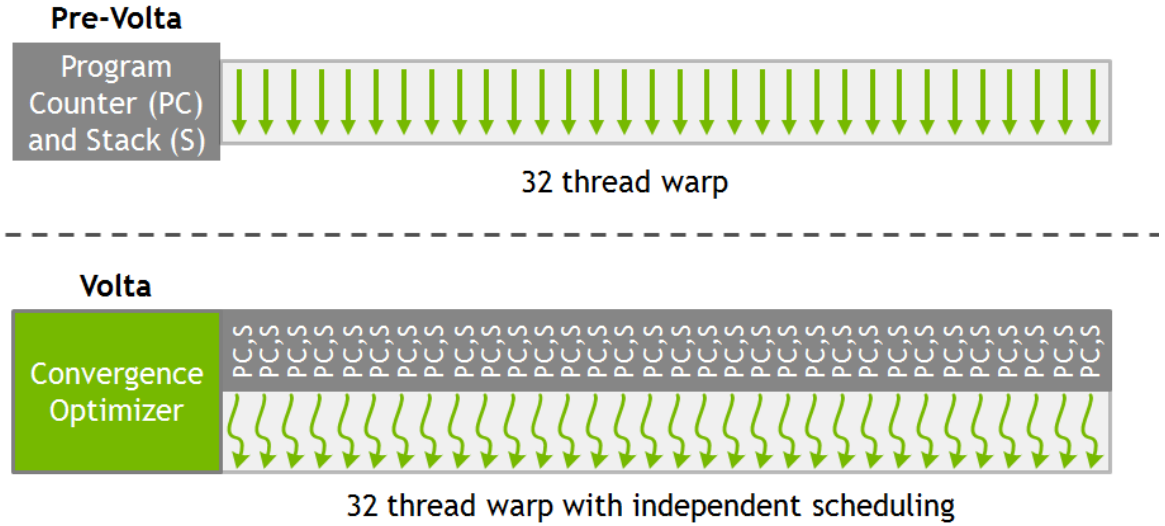
Pascal (2016)

- Introduced NVLink, NVIDIA's high speed, high bandwidth interconnect
  - PCI BW being a bottleneck for multi-GPU systems
  - GPU-to-GPU data transfers at up to 160 GB/s (5x PCI)
- HBM2 for GPU memory
  - Stacked memory with high bandwidth
- Unified memory
  - Provides a single, seamless unified virtual address space for CPU and GPU memory
  - Support 49-bit virtual addressing
  - Memory page fault support → global data coherencecy

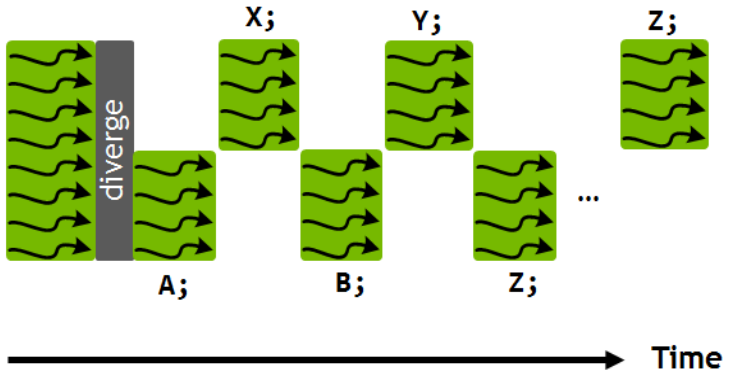


Volta (2017)

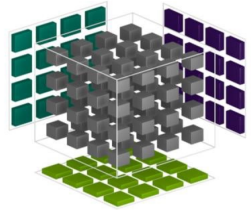
- Independent thread scheduling
  - Maintain execution state per thread, including a PC and call stack
  - Schedule optimizer determines how to group active threads from the same warp
  - Still SIMT: at any given clock cycle, CUDA cores execute the same instruction for all active threads in a warp
- Cooperative Groups: allow synchronizations at sub-block and multiblock granularities




```
if (threadIdx.x < 4) {
    A;
    B;
} else {
    X;
    Y;
}
Z;
```



Tensor Core in Volta



- Specialized compute unit for mixed-precision matrix computation
- Each tensor core completes a 4×4x4 FP16 MMA (matrix-multiply and accumulate) each cycle
  - Ampere: 8x4x8 FP16 MMA (fewer, higher-throughput Tensor Cores)
- Warp-level programming through WMMA and MMA instructions
  - All 32 threads cooperate to perform a matrix operation

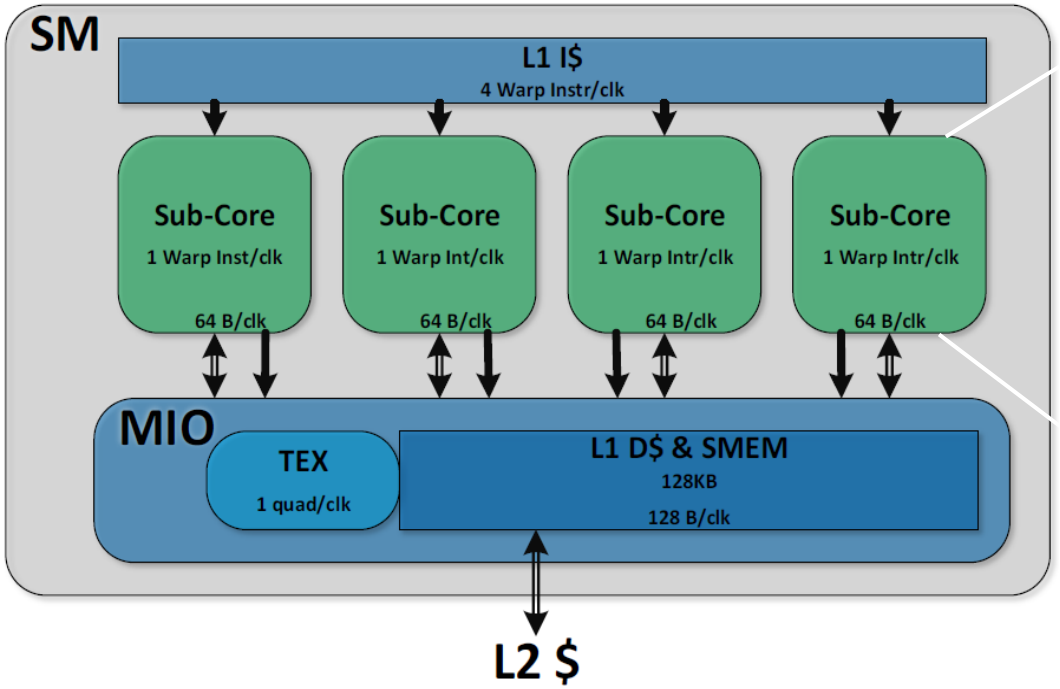
$$\begin{pmatrix} A_{0,0} & A_{0,1} & A_{0,2} & A_{0,3} \\ A_{1,0} & A_{1,1} & A_{1,2} & A_{1,3} \\ A_{2,0} & A_{2,1} & A_{2,2} & A_{2,3} \\ A_{3,0} & A_{3,1} & A_{3,2} & A_{3,3} \end{pmatrix} \begin{pmatrix} B_{0,0} & B_{0,1} & B_{0,2} & B_{0,3} \\ B_{1,0} & B_{1,1} & B_{1,2} & B_{1,3} \\ B_{2,0} & B_{2,1} & B_{2,2} & B_{2,3} \\ B_{3,0} & B_{3,1} & B_{3,2} & B_{3,3} \end{pmatrix} + \begin{pmatrix} C_{0,0} & C_{0,1} & C_{0,2} & C_{0,3} \\ C_{1,0} & C_{1,1} & C_{1,2} & C_{1,3} \\ C_{2,0} & C_{2,1} & C_{2,2} & C_{2,3} \\ C_{3,0} & C_{3,1} & C_{3,2} & C_{3,3} \end{pmatrix}$$

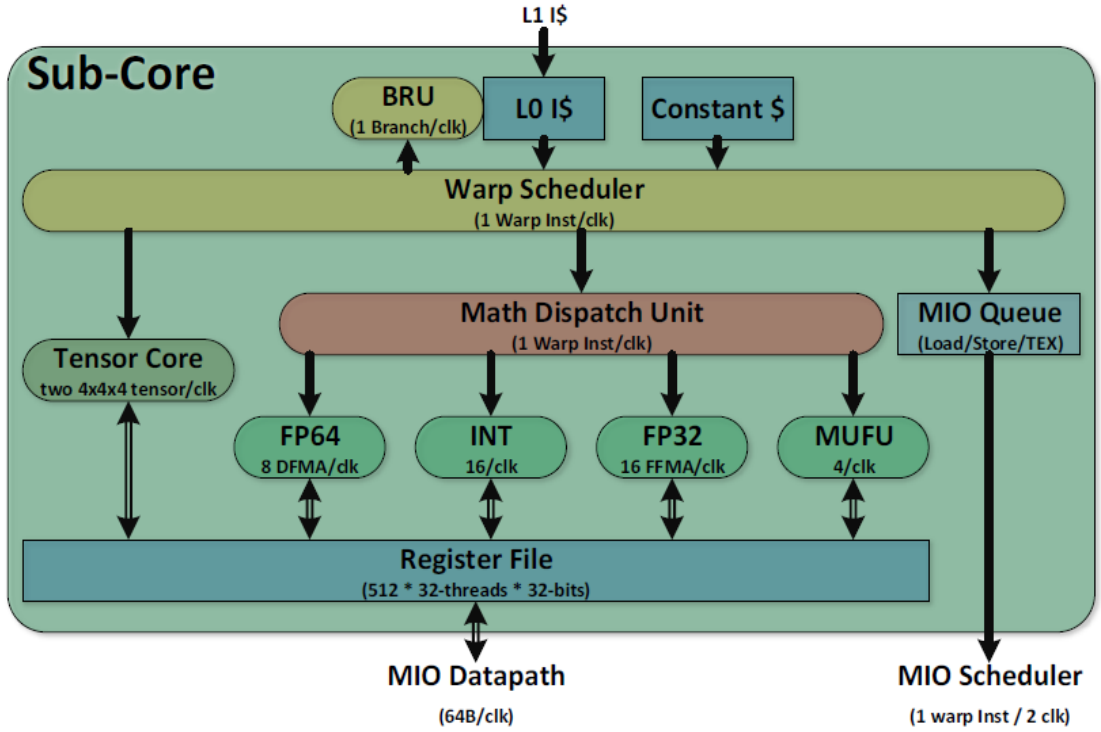
FP16 FP16 FP16 or FP32

$$D = AB + C$$

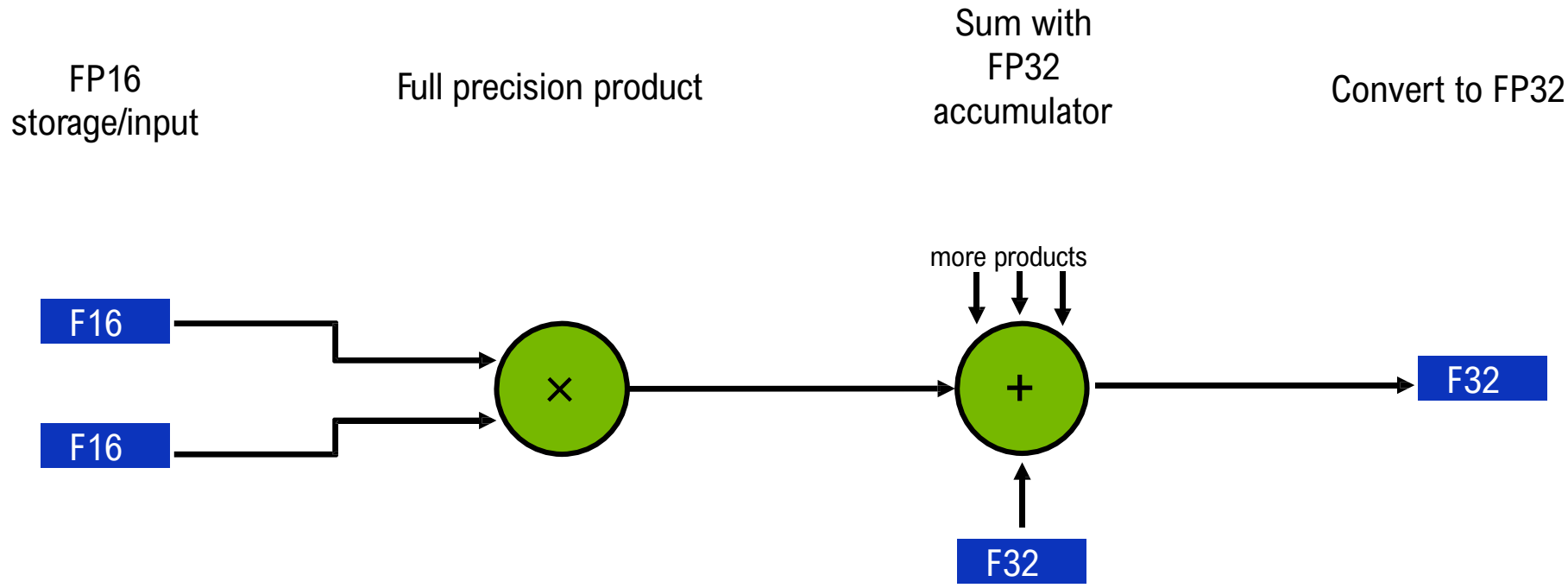
SM Design in Volta

- 4 “Sub-Core”s per SM
  - Two Tensor Cores per “Sub-Core” (One for Ampere)
- L1 instruction/data caches, texture cache, and shared memory per SM
- Warp scheduler per sub-core: warps are statically distributed across sub-cores



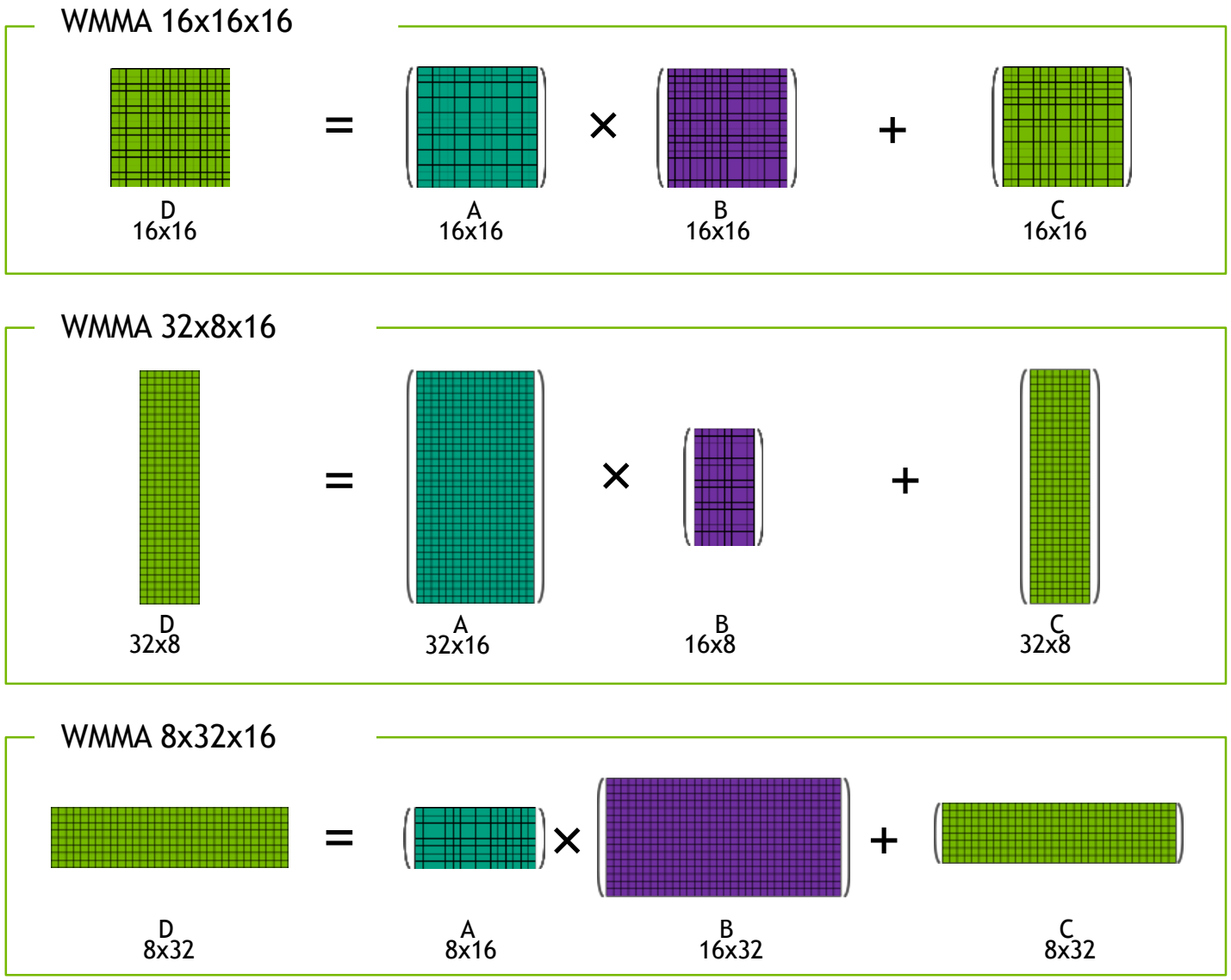


Tensor Operation in Volta



Also supports FP16 accumulator mode for inferencing

Supported Tile Sizes (CUDA 10)



Typical Usage of Tensor Cores

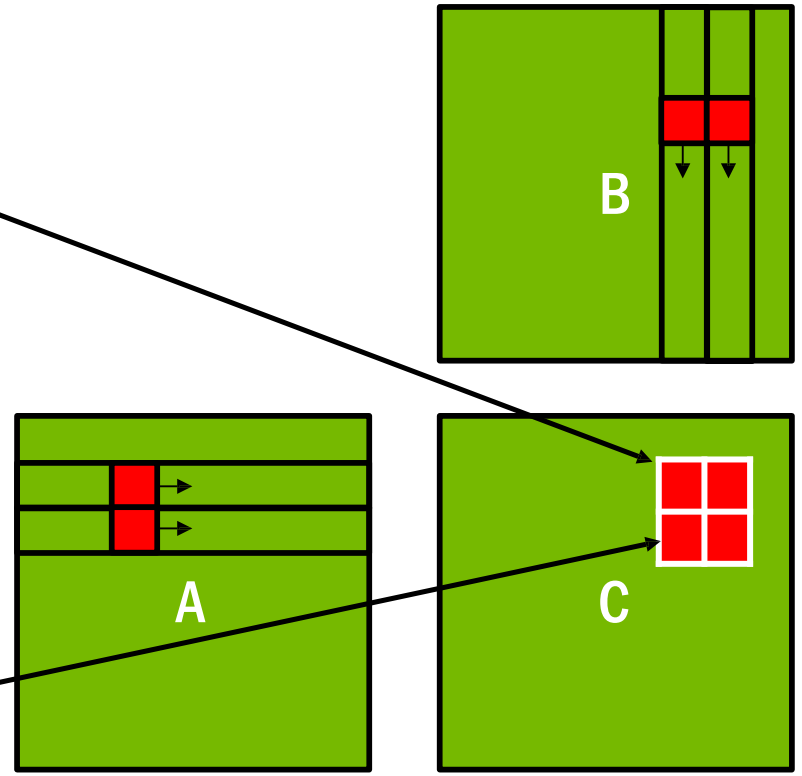
Each warp processes a 16x16 output tile

Each warp:
Loop on all input tiles A_k and B_k

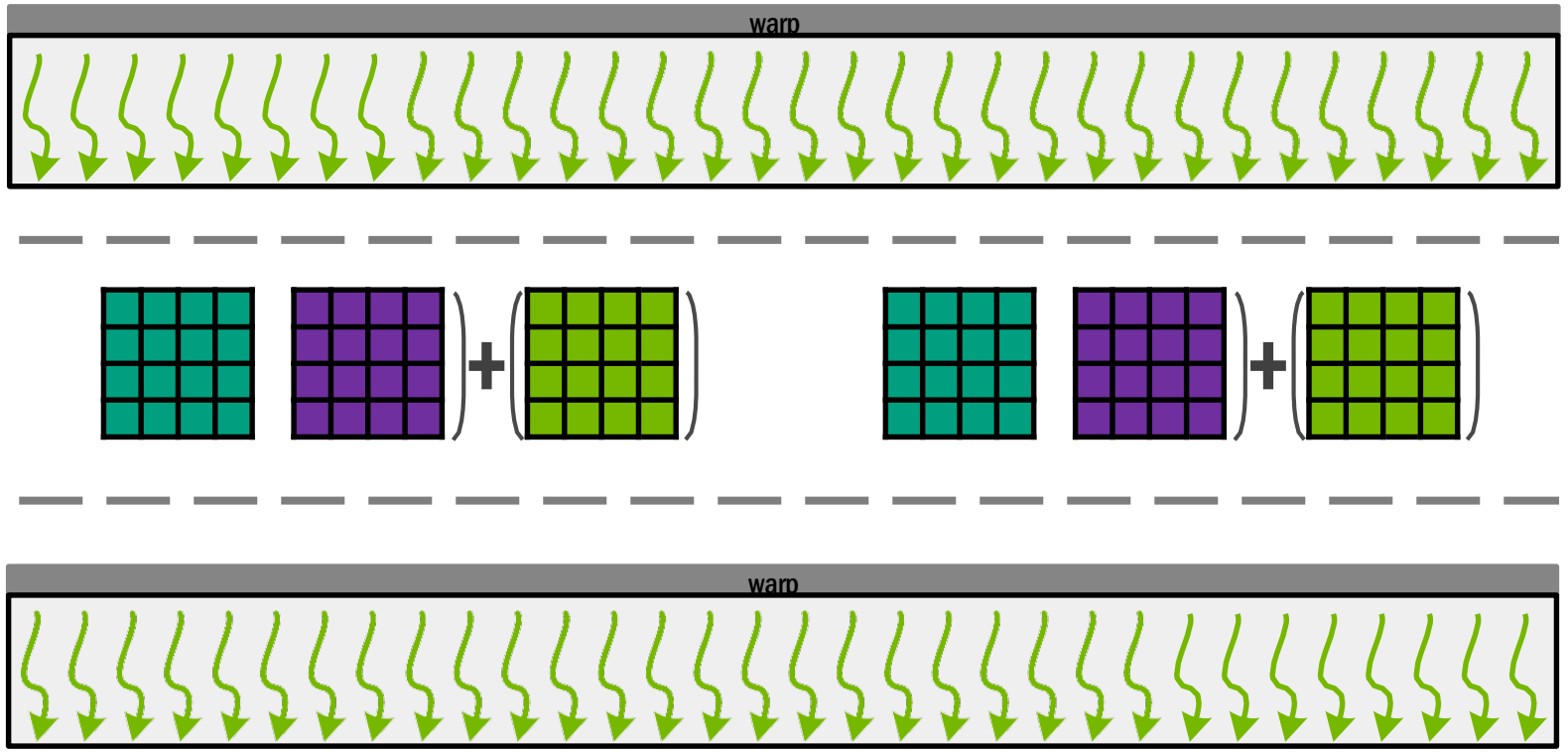
$$C = C + A_k \times B_k$$

Write the output tile

Can compute several tiles per thread block, with inputs staged in shared memory



Tensor Core Coordination



Warp-synchronizing operation for cooperative matrix math

Aggregate Matrix Multiply and Accumulate for 16x16 matrices

Result distributed across warp

Tensor Core Example




```
__device__ void tensor_op_16_16_16(  float *d, half *a, half *b,
        float *c)
{
    wmma::fragment<matrix_a,                ...> Amat;
    wmma::fragment<matrix_b,                ...> Bmat;
    wmma::fragment<matrix_c,                ...> Cmat;

    wmma::load_matrix_sync(Amat, a, 16);
    wmma::load_matrix_sync(Bmat, b, 16);
    wmma::fill_fragment(Cmat, 0.0f);

    wmma::mma_sync(Cmat, Amat, Bmat, Cmat);

    wmma::store_matrix_sync(d, Cmat, 16, wmma::row_major);

}
```

CUDA C++

Warp-Level Matrix Operations

Tensor Core – Volta vs. Turing vs. Ampere

◎ Used via CUBLAS, CUDNN, CUTLASS, TensorRT or CUDA C++ API for WMMA

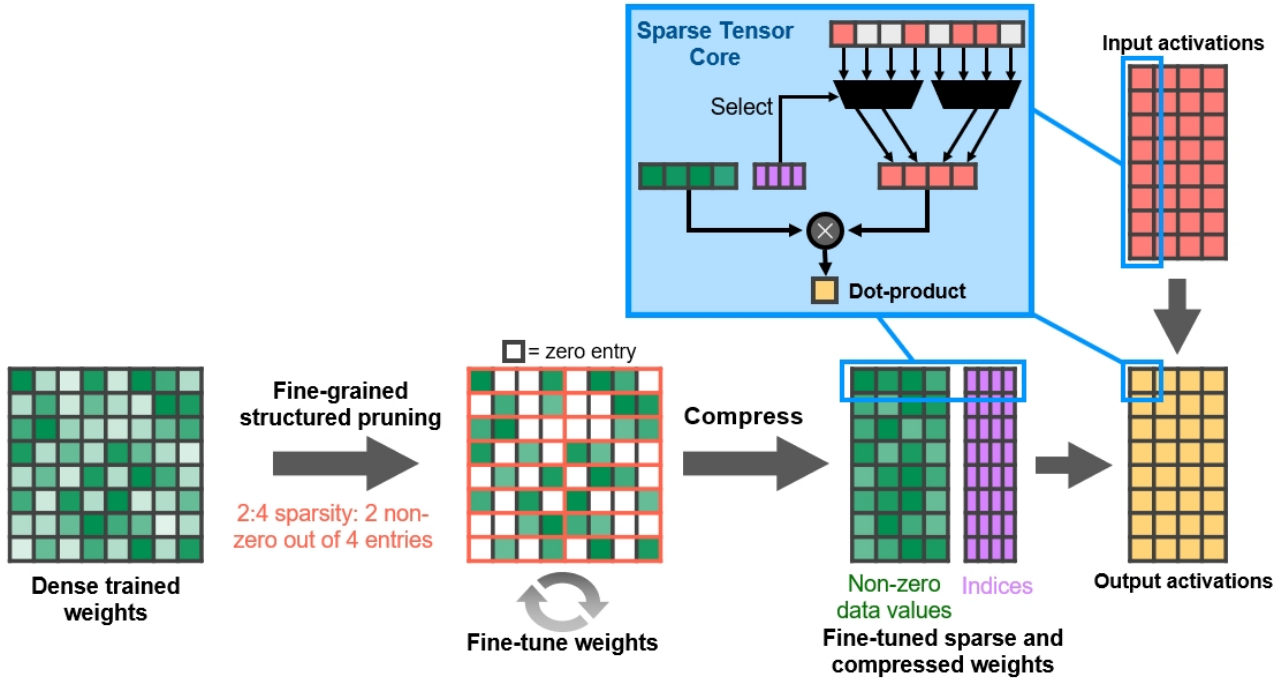
◎ Binaries using Tensor Cores should be recompiled for later generations to achieve full throughput

GPU	SMs	Total	Peak FP16		Peak INT8	Peak INT4	Peak INT1	Peak TF32
V100	80	640	125	TFlops	N.A.	N.A.	N.A.	N.A.
TU102	72	576	130	TFlops	261 Tops	522 Tops	2088 Tops	N.A
A100	108	432	312	TFlops	624 Tops	1248 Tops	4992 Tops	156 Tflops
A100 Sparsity	-	-	624	TFlops	1248 Tops	2496 Tops	N/A	312 Tflops

Structured Sparsity

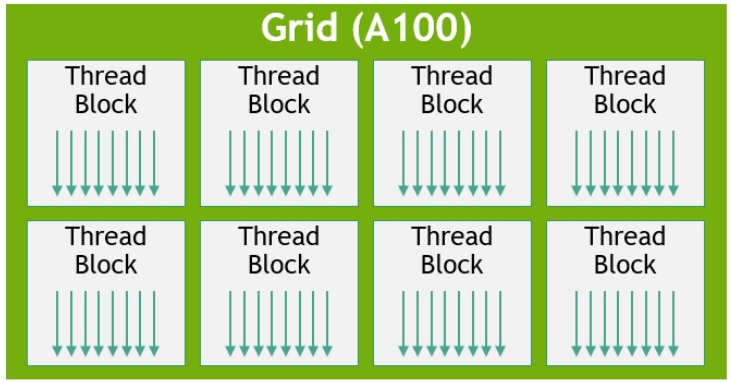
◎ Adapt weights during the training process, enforce the structure constraints of 2:4 sparse matrix

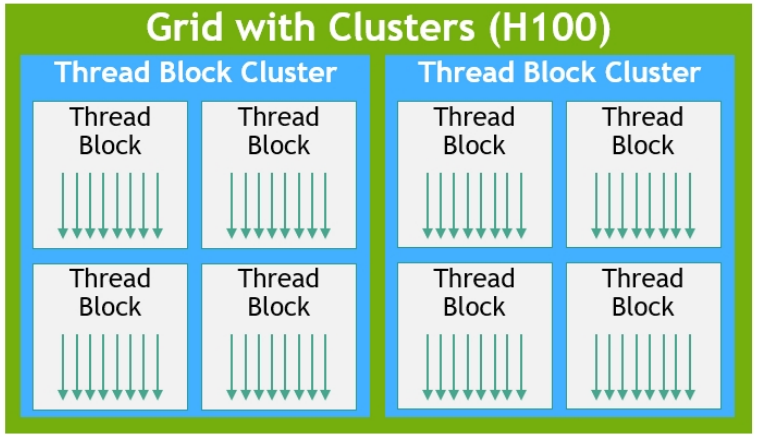
- Two non-zero values in every four-entry vector
- Compressed efficiently and reduce storage and bandwidth by almost 2x



Hopper (2022)

- ”Tensor Core GPU” with 4th-generation Tensor Cores
- Thread block clusters
  - Another level of the programming hierarchy (threads → thread blocks → thread block clusters → grids)
  - Enable multiple thread blocks running concurrently across multiple SMs
- Distributed shared memory: all the threads within a cluster to directly access other SM’s shared memory through dedicated SM-to-SM network





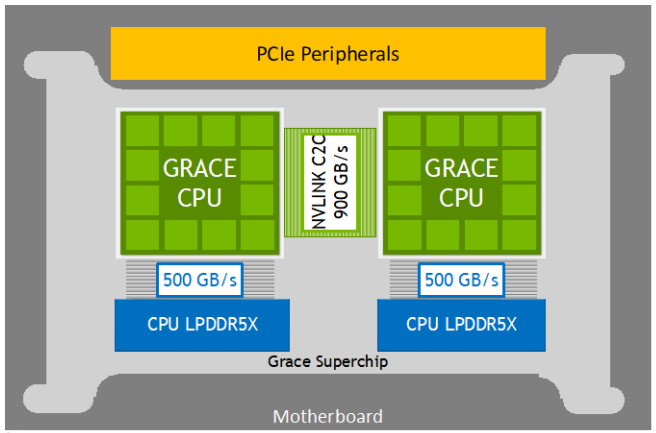
Hopper (2022)

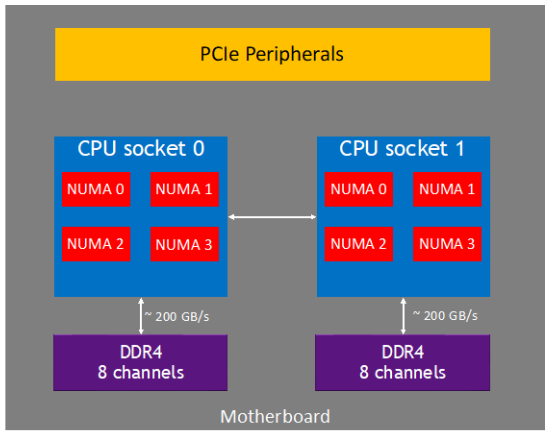
- Tensor memory accelerator (TMA)
  - Improve data fetch efficiency for Tensor Cores
  - Transfer large blocks of data and multi-dimensional tensors from global memory to shared memory and vice-versa
  - Can by asynchronous (threads can execute other independent work)
- Transformer engine: SW + tensor Core for Transformer model training and inference
- Confidential computing support
  - Protects user data, defends against hardware and software attacks
  - Creates a hardware-based Trusted Execution Environment (TEE)

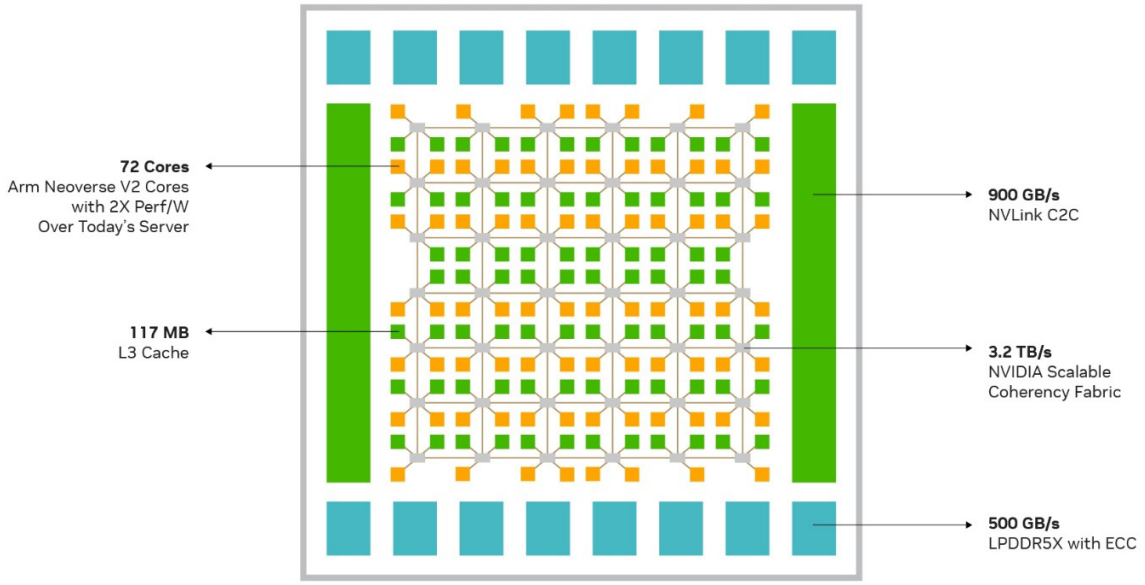
Grace CPU (2023)

◎ ARM CPUs designed to run diverse workloads like AI, data analytics, cloud applications, HPC

- 2 72 ARM cores in a single module
- Connected coherently over NVIDIA NVLink Chip-to-Chip (C2C)
- LPDDR5X memory
- NVIDIA Scalable Coherency Fabric (SCF) is a mesh fabric, distributed cache architecture

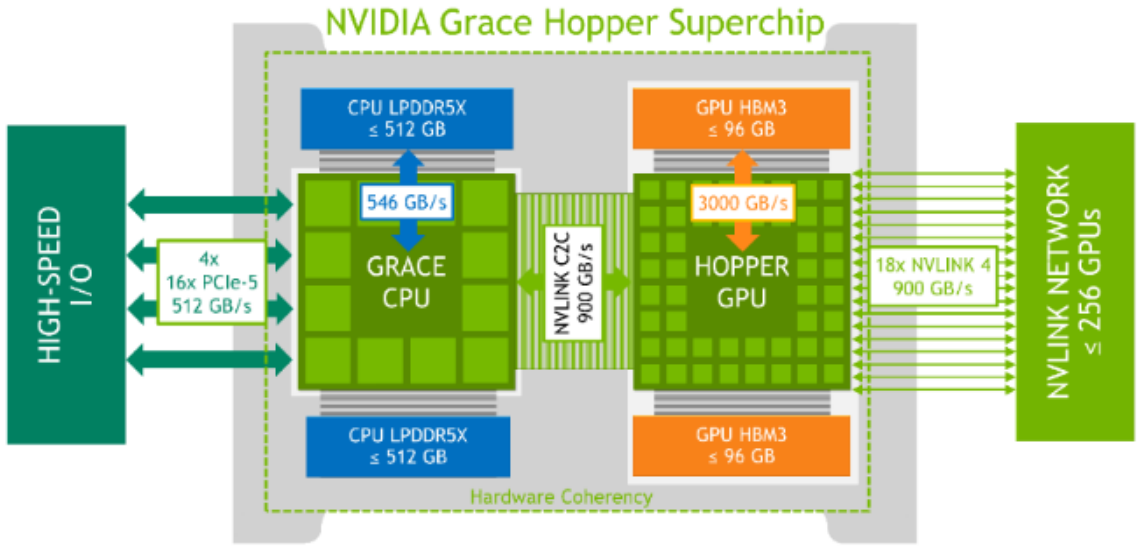






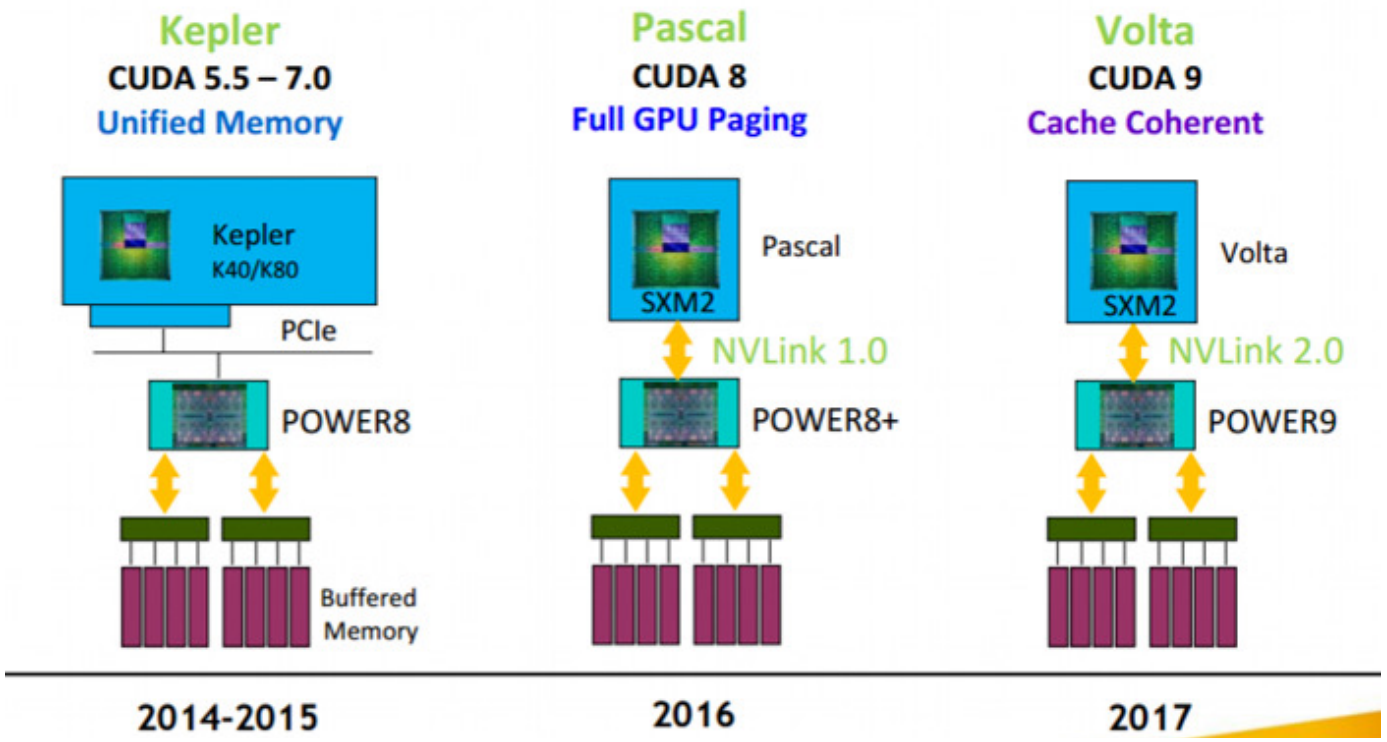
Grace Hopper Superchip (2023)

- NVIDIA's first GPU-CPU superchip
- Hardware-designed memory coherency through NVLink-C2C between CPU and GPU
  - CPU and GPU threads can transparently and concurrently access both CPU & GPU local memory
- NVLink : 900 GB/s per GPU (c.f., 64 or 128 GB/s for PCIe)
  - Formerly NVIDIA GPU exclusive interconnect (mainly), now chip-to-chip communication



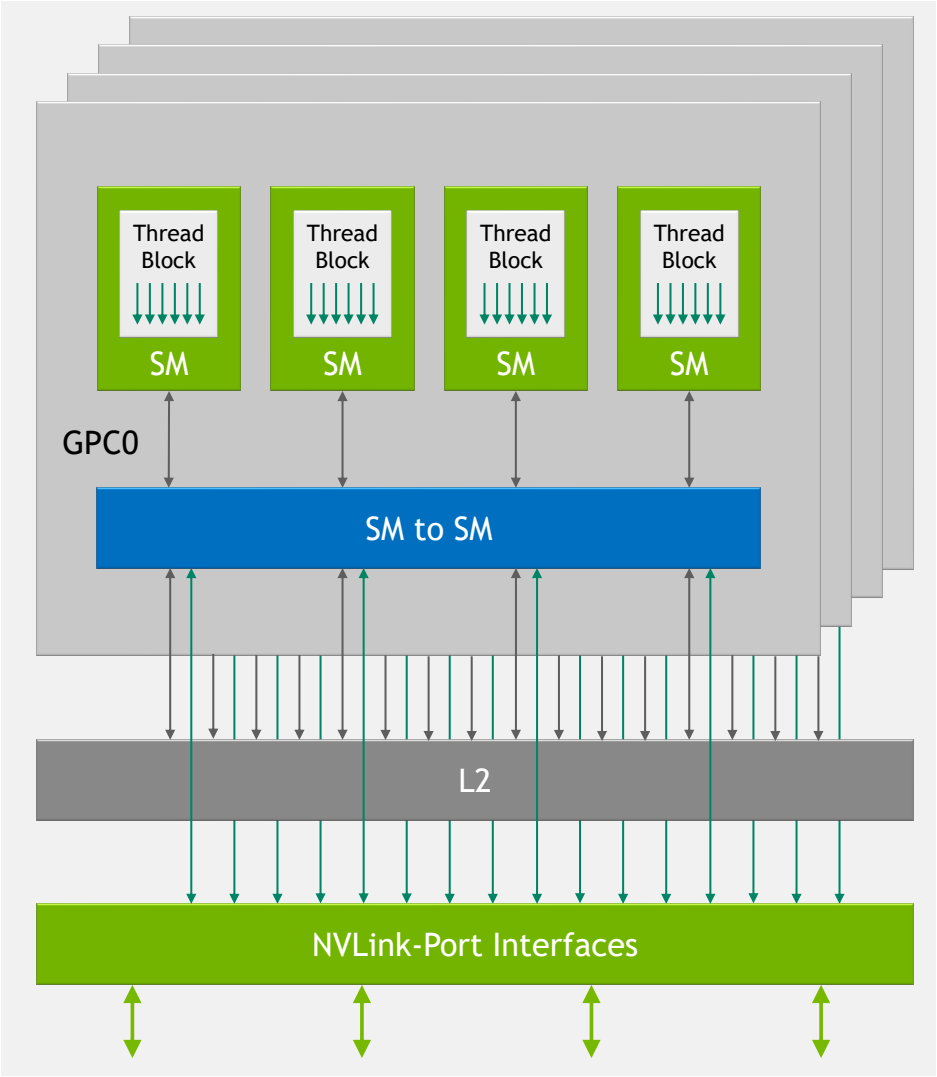
NVLink Overview

- GPU-to-GPU and CPU-to-GPU interconnect
  - Alternate interconnect to PCIe
  - Simplified programming model: NVLink with unified memory



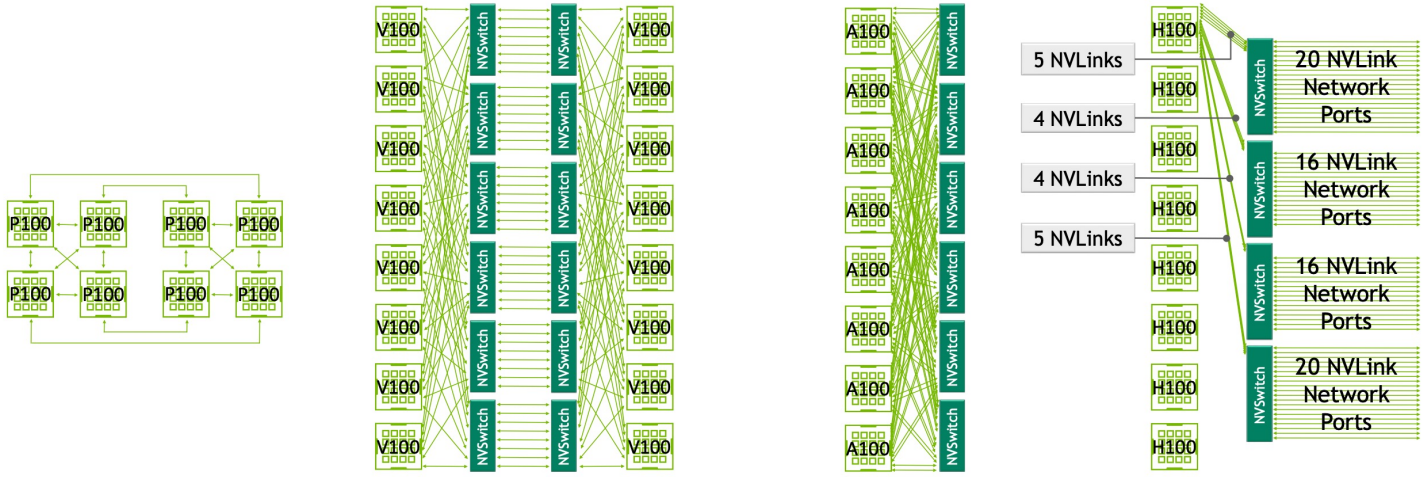
Motivation

- GPU operational characteristics match NVLink spec
  - Thread-block execution structure efficiently feeds parallelized NVLink architecture
  - NVLink-Port interfaces match data-exchange semantics of L2 as closely as possible
- Higher aggregate bandwidth than PCIe
  - 900 GB/s (NVLink 4) vs. 128 GB/s (PCIe 5.0)
  - Multiple NVLinks can be “ganged” to realize higher aggregate lane counts
- Direct data exchange reduces communication overhead
  - Peer-to-peer transfers avoid staging data through CPU memory
- Supports hardware coherence between CPU and GPU
  - NVLink-C2C in Grace Hopper



NVLink-Enabled GPU Servers

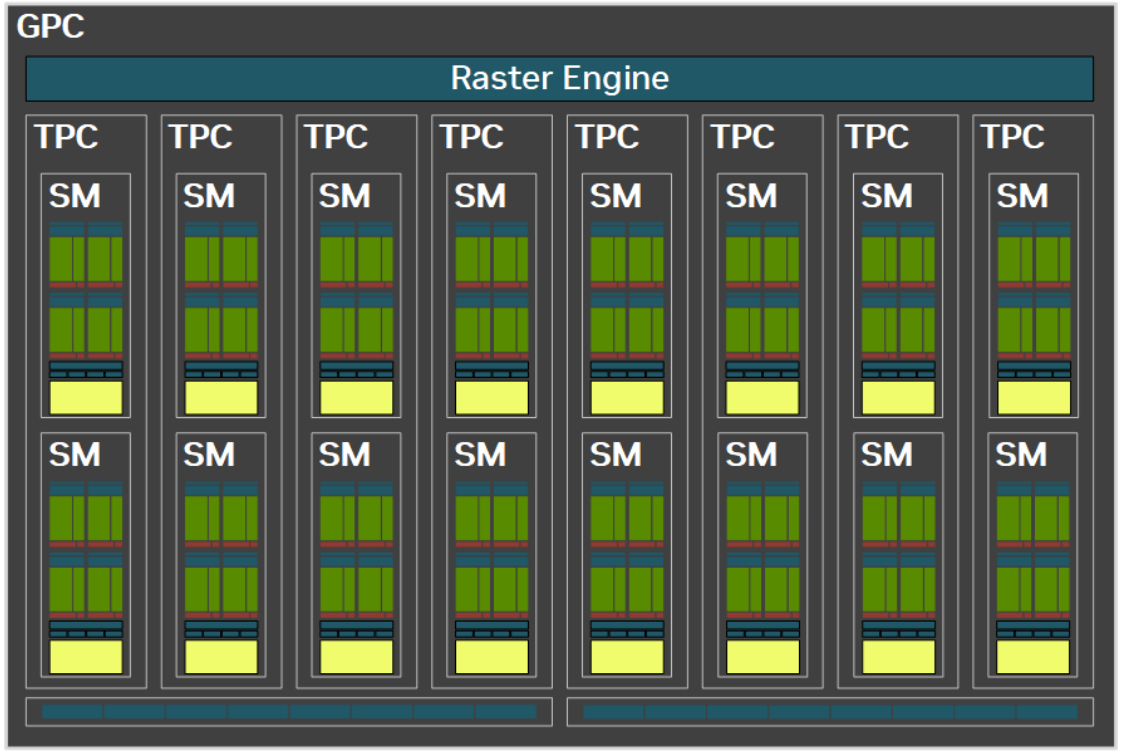
- NVSwitch provides any-to-any connectivity
  - Designed to enable clusters of much larger GPUs by routing GPUs through one or more switches
    - Before NVSwitch, if you wanted a cluster larger than 8 GPUs, need to connect multiple systems via Infiniband or such → Losing some of the shared memory and latency benefits of NVLink and closely connected GPUs
  - Enable any kind of GPU topology with the use of multiple switches



2016	2018	2020	2022
DGX-1 (P100)	DGX-2 (V100)	DGX A100	DGX H100
140GB/s Bisection BW 40GB/s AllReduce BW	2.4TB/s Bisection BW 75GB/s AllReduce BW	2.4TB/s Bisection BW 150GB/s AllReduce BW	3.6TB/s Bisection BW 450GB/s AllReduce BW

Blackwell (2025)

- Deeper hierarchy
  - 12 GPCs (Graphics Processing Clusters), 96 TPCs (Texture Processing Clusters), 192 SMs (Streaming Processors), 24,576 CUDA cores
    - GPC is the high-level hardware block with raster engine, eight TPCs, and 16 SMs
  - Tightly integrated with specialized graphic processing hardware

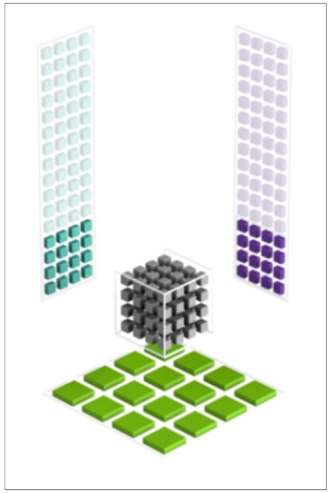


RTX vs. HPC/AI

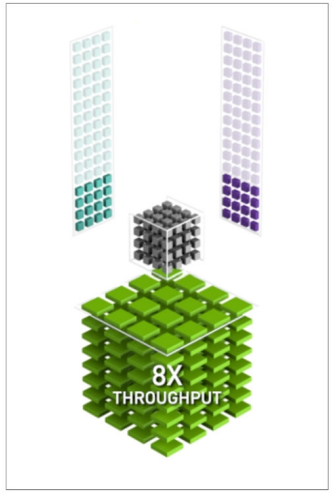
- There are variants optimized with different priorities for target applications
  - Graphics and RTX
    - Optimize for frame latency, rasterization, ray tracing, display
  - HPC and AI
    - Throughput, memory capacity, reliability, scale-up
  - Both share common foundation: SMs, warps, CUDA cores, Tensor Cores, caches, shared memory, etc.

RTX Blackwell

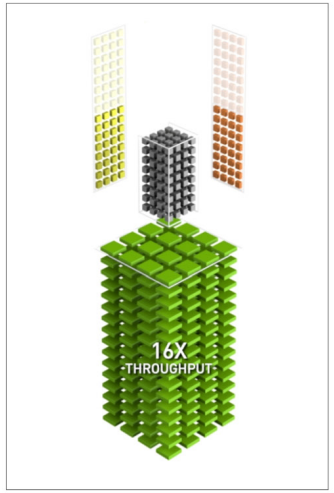
- SM architecture
  - Unified FP32/INT32 CUDA cores
  - Increased Texture units
    - Handles texture filtering and texture coordinates
  - Tensor Cores with support for FP16, BF16, TF32, INT8, FP6, and FP4
    - FP8 Transformer Engine
  - RT cores
    - Ray tracing enables physically accurate lighting, shadows, and reflections



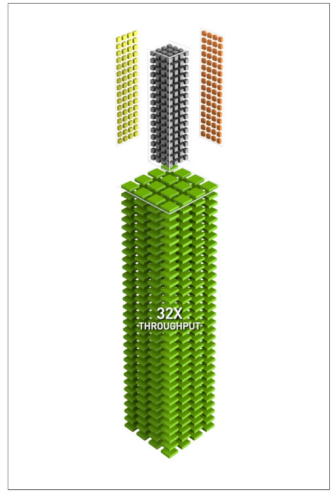
Pascal



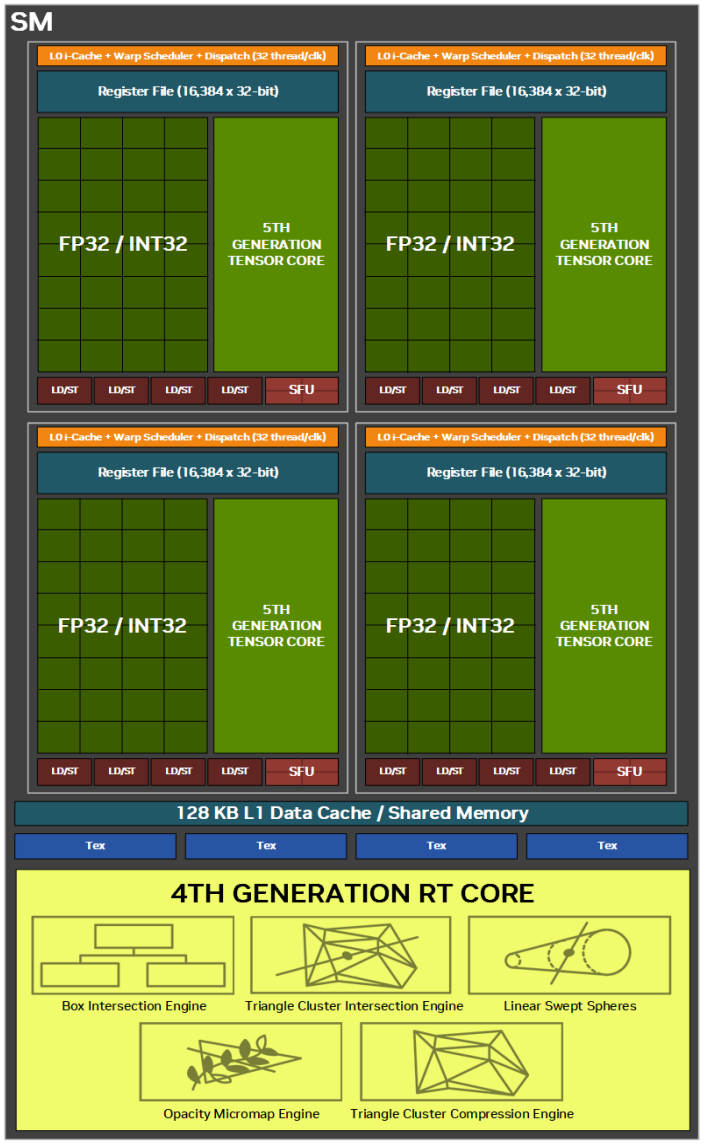
Turing Tensor Core
FP16



Ada Tensor Core
FP8



Blackwell Tensor Core
FP4

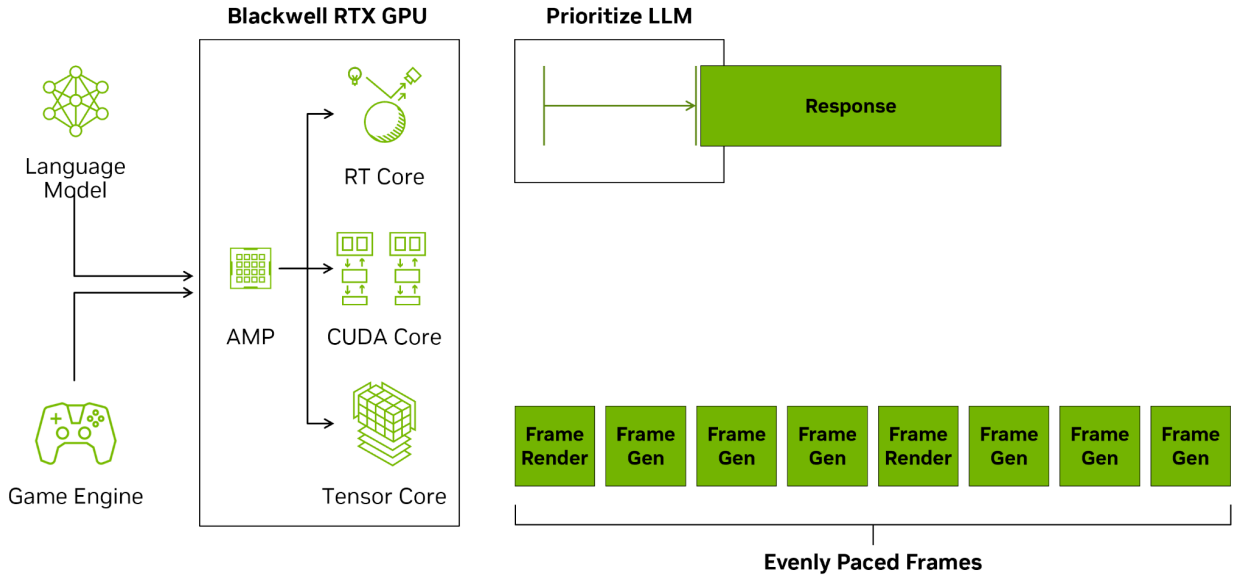


HPC/AI Blackwell

◎ AI Management Processor (AMP)

- Programmable context scheduler designed to offload scheduling of GPU contexts from the system CPU
  - Task isolation, sharing the GPU without conflicts
  - RISC-V processor located at the front of the GPU pipeline

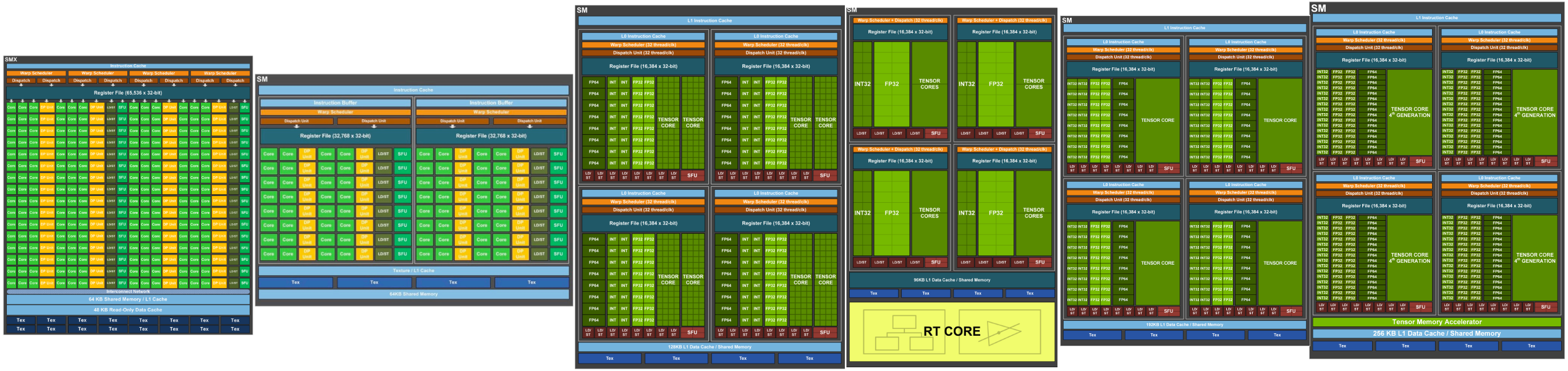
→ Take over the responsibility of the CPU's scheduling of GPU tasks, reducing dependency on the system CPU



Blackwell Example: RTX GB202 and B200

Design dimension	RTX Blackwell (GB202)	Data-center Blackwell (B200)
Primary goal	Interactive graphics and local AI	AI/HPC throughput and scale
Specialized hardware	RT Cores, raster/texture/display, Tensor Cores	Tensor Cores, FP64, Transformer Engine, RAS/decompression
Memory	GDDR7	HBM3e
System fabric	PCIe and display interfaces	NVLink 5 and NVSwitch
Typical software	DirectX, Vulkan, OptiX, CUDA	CUDA libraries, AI frameworks, NCCL, MPI
Optimization focus	Frame time and visual quality	Throughput, memory capacity, reliability, communication

Architectural Evolution



Kepler (2012)	Pascal (2016)	Volta (2017)	Turing (2018)	Ampere (2020)	Hopper (2023)
2688 cores, 15 SMX	3840 cores, 60 SM	5120 cores, 84 SM	4608 cores, 72 SM	8192 cores, 128 SMs	132 SMs
192 FP32, 64 FP64, 32 SFU	64 FP32, 32 FP64, 16 SFU	64 INT32, 64 FP32, 32 FP64, 8 Tensor cores	64 INT32, 64 FP32, 2 FP64, 8 Tensor cores, 1 RT core	64 INT32, 64 FP32, 32 FP64, 4 Tensor cores	64 INT32, 128 FP32, 64 FP64, 4 Tensor cores
4 warp schedulers	2 warp schedulers	4 warp schedulers	4 warp schedulers	4 warp schedulers	4 warp schedulers
64KB L1, 1.5MB L2	64KB SM, 24KB L1, 4MB L2	128KB L1, 6MB L2	96KB L1, 6MB L2	192KB L1, 40MB L2	256KB L1, 50MB L2
Unified memory, dynamic parallelism	High Bandwidth Memory (HBM), NVLink	Tensor cores, NVLink2, independent thread scheduling	RT core	Sparse matrix operation support	Thread block clusters, confidential computing support

Architectural Evolution

- Scaling interconnect and memory bandwidth
  - Addresses CPU-GPU, GPU-GPU communication bottlenecks
  - NVLink and NVSwitch
- More heterogeneity within a chip
  - Tensor cores, RT cores
- More flexible execution model
  - Fine-grained synchronization between threads
  - Synchronizations between thread blocks (”cooperative groops”)
  - Independent thread scheduling to support fine-grained synchronization
- More programmable host-device data communication
  - Support for unified memory (runtime) and hardware coherence (NVLink)
- Increasingly focusing on data center and cluster support
  - Security features for virtual machines on GPU

Next Class

GPU Architectures 2